

Programación del microcontrolador ATmega328 y comunicación con la PC

Gonzalo F. Perez Paina

7 de julio de 2025

Programación del microcontrolador ATmega328 y comunicación con la PC by Dr. Ing. Gonzalo F. Perez Paina is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.



Índice general

1. Programación del microcontrolador ATmega328	5
1.1. Introducción	5
1.2. La placa Arduino UNO	5
1.3. El microcontrolador ATmega328	6
1.3.1. Memoria	7
1.3.2. Puerto digital de entrada/salida	8
1.3.3. Puerto serie USART	10
1.4. Programación del ATmega328	14
1.4.1. Programación con el IDE Arduino	14
1.4.2. Programación sin el IDE Arduino	17
1.5. Programación de los puertos digitales	19
1.5.1. Programación de salidas digitales	19
1.5.2. Programación de entradas digitales	20
1.5.3. Constantes simbólicas y macros	21
1.6. Programación de la UART	23
1.6.1. Configuración de la USART	23
1.6.2. Transmisión de datos	24
1.6.3. Recepción de datos	24
2. Comunicación serie	31
2.1. Introducción	31
2.2. Comunicación paralela y serie	31
2.3. Transmisión síncrona vs. asíncrona	32
2.4. Comunicación Half-Duplex y Full-Duplex	33
2.5. Ejemplos de comunicación paralela y serie	34
3. Estándar RS-232	35
3.1. Introducción	35
3.2. Características eléctricas	36
3.3. Características funcionales	37
3.4. Características mecánicas	37
3.5. Receptor/transmisor asíncrono universal, UART	37
4. Puerto serie en una PC	39
4.1. Introducción	39
4.2. Acceso a dispositivos	39
4.2.1. Dispositivos TTY	40
4.3. Comunicación entre dispositivos	40
4.3.1. Cable null-model	40
4.3.2. Utilidad <code>socat</code>	41
4.4. Programación del puerto serie de la PC	42
4.4.1. Programa ejemplo	42
4.4.2. Configuración de la terminal – estructura <code>termios</code>	43
4.4.3. Módulo de software para configuración del puerto serie	43

Capítulo 1

Programación del microcontrolador ATmega328

1.1. Introducción

Se describe la programación del microcontrolador ATmega328 incluido en varias placas Arduino, en particular la placa Arduino UNO, haciendo principal énfasis en la programación de los puertos digitales de entrada/salida y de la comunicación serie asíncrona [1]. Se comienza con una breve descripción de la placa Arduino UNO para luego centrar la atención en el microcontrolador (μC) que esta incluye, el ATmega328 [2]. Se hace mención también a los tipos de memoria presentes en este μC y dos de los periféricos más importantes: i) los puertos digitales de entrada/salida y ii) el puerto de comunicación serie asíncrona (UART).

Desde el punto de vista de la programación primero se presenta cómo programar el μC utilizando el IDE Arduino para luego pasar a la programación sin IDE, que será el método utilizado en el resto del documento. Finalmente, se muestran varios ejemplos de programación de los periféricos de interés.

Para los ejemplos de programas del μC ATmega328 se utilizará la placa Arduino UNO conectada a la placa de expansión (o Shield, en la jerga de Arduino) “ponchitoCIII” [3]. Se puede utilizar también su versión en el simulador TinkerCAD [4].

1.2. La placa Arduino UNO

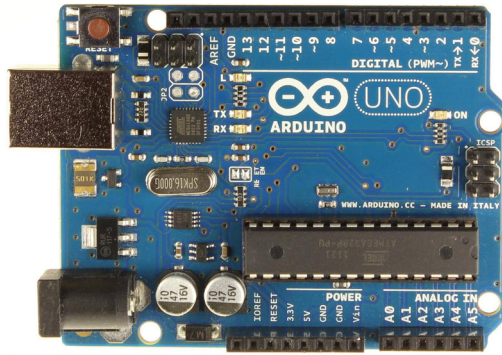
La placa Arduino UNO es el primer modelo desarrollado por Arduino.cc [5] y es la placa más utilizada cuando se comienza a programar en el mundo Arduino [6]. La figura 1.1 muestra dos versiones de placa Arduino UNO más comunes: Arduino UNO Rev3 (original) en la figura 1.1a y Arduino UNO CH340 en la figura 1.1b. La principal diferencia entre estas dos versiones es el conversor USB/serie utilizado para la comunicación con la PC cuyo principal propósito es cargar el programa en el microcontrolador.

La placa Arduino UNO Rev3 [7] tiene un microcontrolador (μC) marca Microchip® (antiguamente Atmel®) modelo ATmega328 [2]. Algunas de las características de esta placa son:

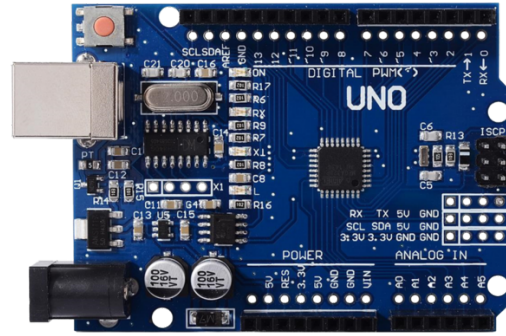
- Tensión de funcionamiento de 5V.
- Tensión de entrada de 7 a 12V (de 6 a 20V como rango máximo).
- 14 pines digitales de entrada/salida.
- 6 pines digitales con función PWM.
- 6 pines de entrada analógica.

El conversor USB/serie de la placa Arduino UNO Rev3 (figura 1.1a) está basado en el μC ATmega16U2 mientras que la placa Arduino UNO CH430 (figura 1.1b) utiliza el circuito integrado (CI) dedicado CH430.

La figura 1.2 muestra el pinout o distribución de pines de la placa Arduino UNO.



(a) Arduino UNO Rev3



(b) Arduino UNO CH430

Figura 1.1: Diferentes versiones de placa Arduino UNO.

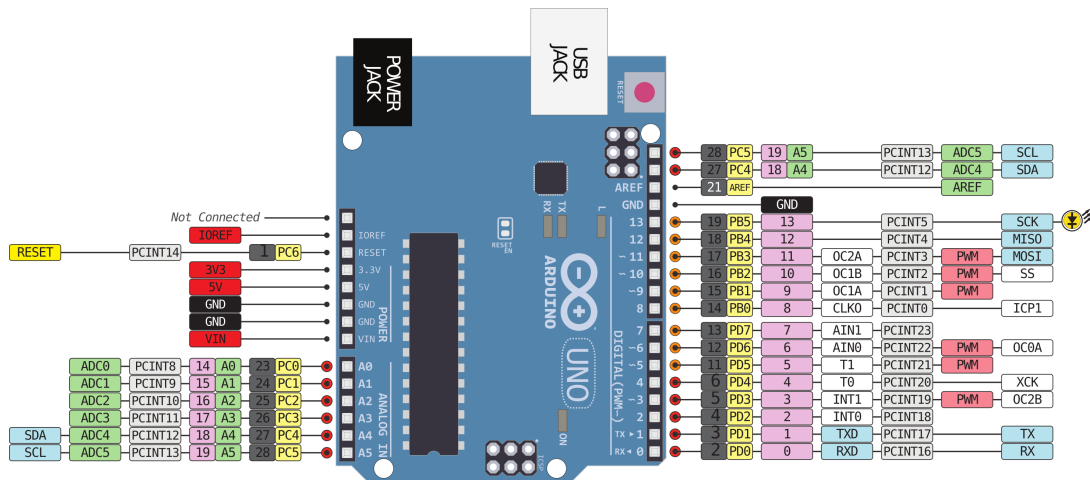


Figura 1.2: Pinout de la placa Arduino UNO.

1.3. El microcontrolador ATmega328

El microcontrolador ATmega328 es un μC de 8-bits de arquitectura AVR RISC mejorado. Algunas de las características de este μC son:

- Arquitectura RISC avanzada:
 - 131 instrucciones (la mayoría de las cuales se ejecutan en un ciclo de reloj).
 - 32 registros de propósitos generales de 8-bits.
 - Máximo de 20MIPS (million instructions per second) @ 20MHz de frecuencia de reloj.
 - Multiplicación por hardware.
- Memoria:
 - 32KB de memoria FLASH.
 - 2KB de memoria SRAM (Static Random Access Memory).
 - 1KB de memoria EEPROM (Electrically Erasable Programmable Read-Only Memory).
- Periféricos:
 - 23 entrada/salida de propósito general (GPIO: General Purpose Input-Output).
 - 6 canales de PWM (Pulse Width Modulation).

- 6/8 canales de ADC (Analog to Digital Converter) de 10-bits.
- Puerto serial programable (USART: Universal Synchronous/Asynchronous Receiver/Transmitter).
- Interfaz serial SPI (Serial Peripheral Interface) maestro-esclavo.
- Interfaz serial de 2-cables (compatible con el I²C de Philips).
- Interrupción externa por cambio de nivel en entrada digital.

La figura 1.3 muestra dos de los encapsulados disponibles del μ C ATmega328: PDIP (Plastic Dual-In Line Package) de 28 pines en la figura 1.3a y TQFP (Thin Quad Flat Package) de 32 pines en la figura 1.3b.

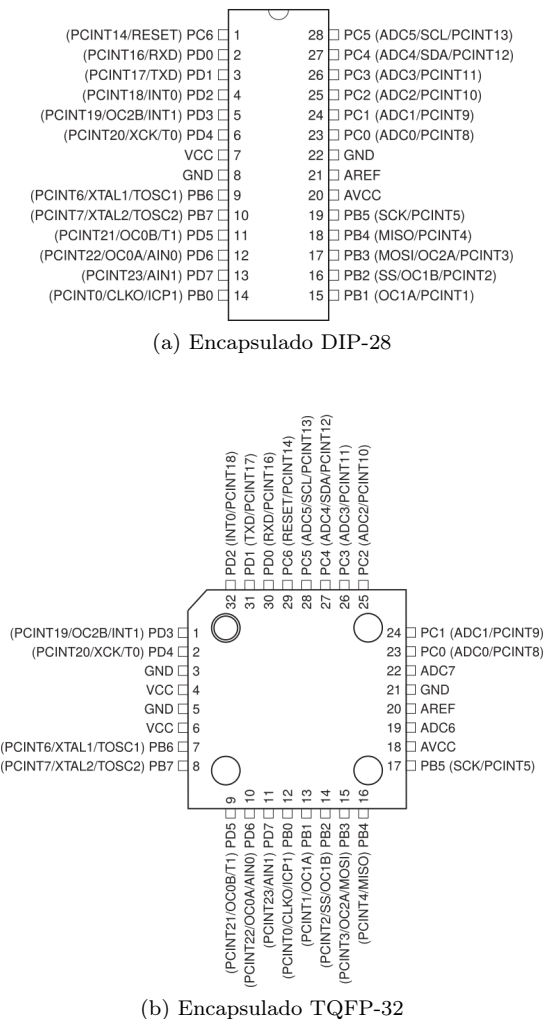


Figura 1.3: Encapsulados más comunes del μ C ATmega328.

Como se observa en la figura 1.3, la mayoría de los pines del μ C pueden multiplexar diferentes funciones de los periféricos disponibles. Por ejemplo: el pin nro. 2 del encapsulado PDIP puede actuar como bit 0 del Puerto D o como señal RXD de la UART o puerto serie.

1.3.1. Memoria

El μ C ATmega328 tiene dos espacios de memorias diferentes: una es la memoria de programa y la otra la memoria de datos. Además, dispone de una memoria no volátil tipo EEPROM (Electrically Erasable Programmable Read-Only Memory) para almacenar datos. Las tres memorias se organizan en espacios lineales y regulares.

La memoria Flash, para almacenar el programa en el μC ATmega328, es de $16\text{K} \times 16\text{-bits}$ haciendo un total de 32KBytes ($32 \times 8\text{-bits}$) y tiene una capacidad de escritura y borrado de al menos 10.000 ciclos. El contador de programa (PC: Program Counter) es de 14-bits lo que permite direccionar 16K ubicaciones de la memoria de programa. La figura 1.4 muestra la organización (mapa de memoria) de la memoria de programa Flash del μC ATmega328.

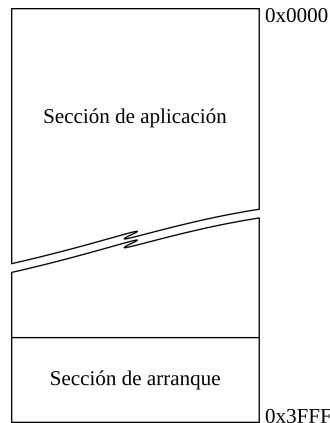


Figura 1.4: Memoria de programa (Flash) del μC ATmega328.

La memoria RAM se organiza en diferentes secciones. Las primeras direcciones están reservadas para acceder a los registros de propósito general y los de entrada/salida y las siguientes para almacenar las variables del programa. Esta última sección tiene un total de 2KBytes ($2\text{K} \times 8\text{-bits}$). La figura 1.5 muestra la organización (mapa de memoria) de la memoria de datos RAM del μC ATmega328.

32 registros	0x0000 - 0x001F
64 registros de E/S	0x0020 - 0x005F
160 registros de E/S ext.	0x0060 - 0x00FF
	0x0100
Memoria RAM gral (2048 x 8-bits)	0x08FF

Figura 1.5: Memoria de datos (RAM) del μC ATmega328.

1.3.2. Puerto digital de entrada/salida

Como se mencionó, la mayoría de los pines del μC ATmega328 pueden multiplexar diferentes funciones (de ahí el nombre de GPIO por General Purpose Input/Output) de los distintos periféricos, siendo una de ellas las de actuar como entrada/salida digital. En esta sección se muestra cómo manejar los puertos de entrada/salida digitales del μC ATmega328.

Registros

Para el manejo de los puertos digitales de entrada/salida se utilizan 3 registros: DDRx , PORTx y PINx donde la x tiene que ser reemplazada por un puerto en particular, o sea B, C o D para el μC ATmega328. Por ejemplo, para el manejo del puerto D se utilizan los registros DDRD , PORTD y PIND . Las funciones de estos registros son:

- DDRx : registro de dirección (Data Direction Register) del puerto x (B, C o D).
- PORTx : registro de datos (Data Register) del puerto x (B, C o D).

- **PINx**: registro de lectura (Input Pin Register) del puerto x (B, C o D)¹.

Los registros de manejo de los puertos digitales de entrada/salida se encuentran mapeados en la memoria RAM de μC (ver figura 1.5), o sea que tienen direcciones de memorias fijas. La tabla 1.1 muestra las direcciones de memoria de los registros para el manejo de los puertos B, C y D del μC ATmega328.

	PINx	DDRx	PORTx
Puerto B	0x23	0x24	0x25
Puerto C	0x26	0x27	0x28
Puerto D	0x29	0x2A	0x2B

Tabla 1.1: Dirección de los registros de manejo de las entradas/salidas digitales.

Algunas de las características eléctricas de los puertos digitales de entrada/salida son: los búfer de las salidas digitales de todos los pines tienen características de corriente simétrica lo que significa que pueden drenar o suministrar la misma corriente eléctrica; y, configurados como salida, los pines tienen suficiente capacidad de corriente como para manejar displays LED de forma directa.

Cabe mencionar que el único puerto disponible en su totalidad (8 bits) en la placa Arduino UNO es el Puerto D (ver figura 1.2). Si bien el Puerto B también está disponible en su totalidad tanto en la versión DIP como SMD del μC , este no se encuentra disponible en los pines de la placa debido a que están multiplexados con los pines para el oscilador a cristal de cuarzo (PB6 y PB7).

La figura 1.6 muestra los registros para el manejo de los puertos digitales de entrada/salida. El μC ATmega328 dispone de un conjunto de estos 3 registros (**DDRx**, **PORTx** y **PINx**) para cada uno de los puertos disponibles. En la figura se observa el nombre de cada registro y de los bits que los componen, si aceptan la operación de lectura y/o escritura y el estado inicial. La x minúscula representa la letra de numeración para el puerto y la n minúscula el número de bit.

Bit	7	6	5	4	3	2	1	0	
	DDx7	DDx6	DDx5	DDx4	DDx3	DDx2	DDx1	DDx0	DDRx
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	
	PORTx7	PORTx6	PORTx5	PORTx4	PORTx3	PORTx2	PORTx1	PORTx0	PORTx
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	
	PINx7	PINx6	PINx5	PINx4	PINx3	PINx2	PINx1	PINx0	PINx
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

Figura 1.6: Registros para el manejo de puertos digitales de entrada/salida.

Los bits **DDxn** del registro **DDx** seleccionan si el pin correspondiente actuará como entrada o salida. Si se escribe un 1 lógico a **DDxn** el pin **Pxn** se configura como salida. Si se escribe un 0 lógico a **DDxn**, el pin **Pxn** se configura como entrada.

Si se escribe un 1 lógico en **PORTxn** cuando el pin está configurado como salida, el pin del puerto se pondrá en alto (uno). Si se escribe un 0 lógico en **PORTxn** cuando el pin está configurado como salida, el pin del puerto se pondrá en bajo (cero). Si se escribe un 1 lógico a **PORTxn** cuando el pin está configurado como entrada, se activará la resistencia de pull-up. Para desactivar (desconectar) la resistencia de pull-up se debe escribir un 0 lógico a **PORTxn** o bien configurar el pin como salida.

Se puede leer el valor del bit de un puerto con **PINxn**, independientemente del valor que tenga el bit **DDxn** correspondiente. Al escribir un 1 lógico en **PINxn** cambiará el valor (toggle) de **PORTxn** correspondiente, independientemente del valor de **DDxn**.

¹Escribir un uno lógico en **PINxn** alterna el valor de **PORTxn**, independientemente del valor de **DDRxn**.

1.3.3. Puerto serie USART

El microcontrolador ATmega328 posee un periférico denominado USART, por las siglas en inglés de Universal Synchronous and Asynchronous Receiver and Transmitter que permite la comunicación serie con otro dispositivo, como por ejemplo una PC. La USART del ATmega328 se denomina USART0 dado que otros modelos de μC pueden tener más de una USART (USART0, USART1, etc.).

En esta sección se describe la USART0 del μC ATmega328 para la comunicación serie asíncrona. Para este tipo de comunicación se utiliza un pin digital para la salida de los datos TXD y un pin digital para la entrada de los datos RXD (ver figura 1.3). Otro tipo de comunicación soportada por la USART del μC ATmega328 es el modo SPI (Serial Peripheral Interface) Maestro.

Algunas de las características de la USART del μC ATmega328 son:

- Permite operación Full-duplex (tiene registros independientes de transmisión y recepción).
- Permite operación síncrona (modo SPI) y asíncrona.
- Soporta tramas seriales con 5, 6, 7, 8 y 9 bits de datos, y 1 o 2 bits de parada.
- Generación de paridad impar o par y verificación de paridad por hardware.

El término “Duplex” en telecomunicaciones se utiliza para definir un sistema que permite una comunicación bidireccional (en ambos sentidos), y Full-duplex significa que la comunicación bidireccional puede darse de forma simultánea.

Trama de comunicación

La trama (frame) de comunicación de la USART en operación asíncrona consiste en una palabra de entre 5 y 9 bits de datos que se envían o reciben de forma serializada. La trama se muestra de forma esquematizada en la figura 1.7.

Todas las tramas comienza con un bit de inicio seguido por el bit de datos menos significativo (LSb, Least Significant bit), luego se envían los siguientes bits de datos hasta un total de nueve, finalizando con el bit más significativo (MSb, Most Significant bit). Si está habilitado, se inserta un bit de paridad luego de los bits de datos y antes del bit de parada. Cuando se completa una transmisión puede seguir de forma inmediata una nueva trama o la línea de comunicación se puede fijar a estado ocioso (idle).

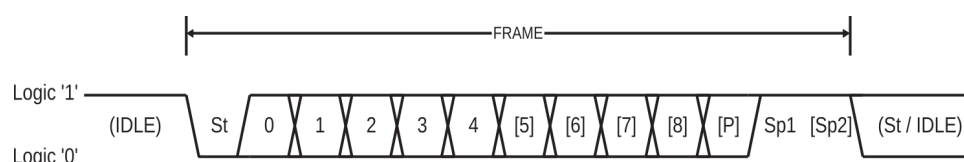


Figura 1.7: Trama (frame) de comunicación serie asíncrona.

La nomenclatura utilizada en la figura 1.7 es:

- **St:** Bit de inicio de trama (nivel bajo).
- **(n):** Bits de datos (0 a 8).
- **P:** Bit de paridad, que puede ser impar o par.
- **Sp:** Bit de parada (nivel alto).
- **IDLE:** No hay transferencia de datos sobre las líneas de comunicación, RXD o TXD (nivel alto).

Diagrama en bloques

El diagrama en bloques de la USART del μC ATmega328 se muestra en la figura 1.8. Como se observa, consiste de 3 bloques principales: i) el generador de reloj (clock), ii) el bloque de transmisión y iii) el de recepción. También pueden observarse los pines TxDn, RxDn y el pin XCKn el cual se utiliza únicamente cuando se activa el modo de transmisión síncrona. Los registros accesibles desde la CPU (remarcados en negrita) son:

- **UDRn**: Registros búfer de transmisión y recepción de datos (USART Data Registers).
- **UBRRn[H:L]**: Registros de velocidad (baud-rate) de transmisión (USART Baud Rate Registers).
- **UCSRnA**, **UCSRnB**, **UCSRnC**: Registros de estado y control (USART Control and Status Registers).

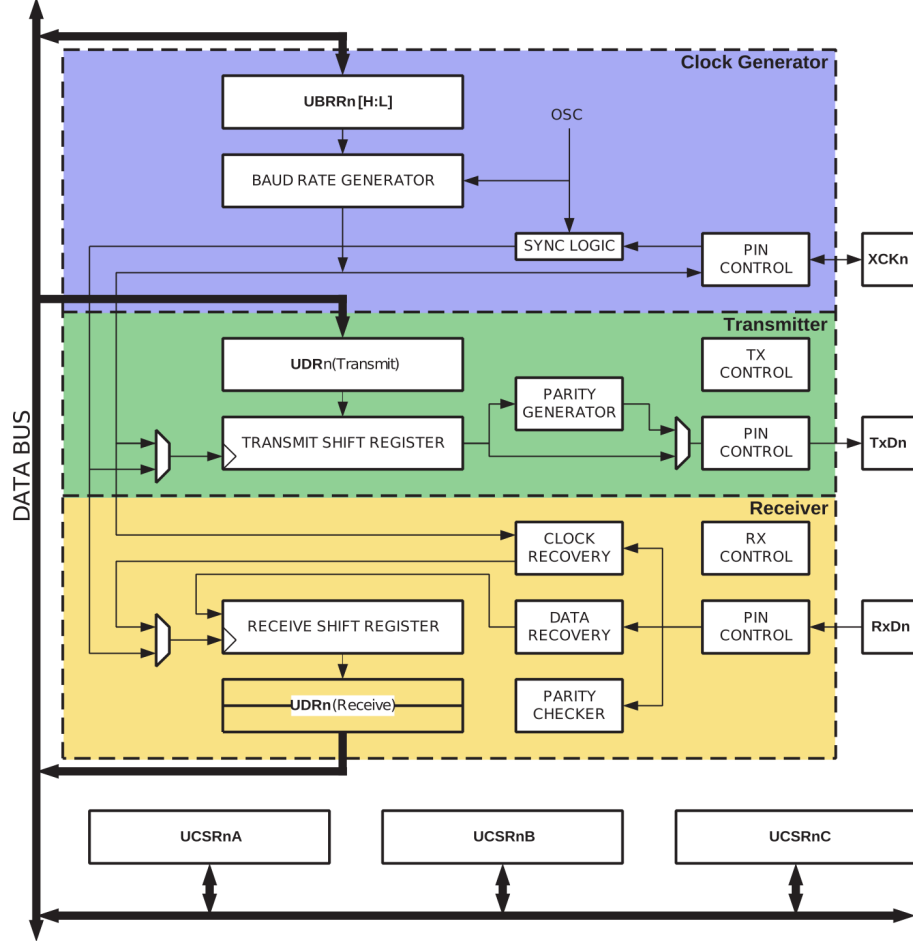


Figura 1.8: Diagrama en bloques del módulo USART.

La lógica para la generación de reloj genera los pulsos de reloj (clock) base para el transmisor y el receptor. La USART soporta 4 modos de operación: *i*) asíncrono normal, *ii*) asíncrono de doble velocidad, *iii*) síncrono maestro y *iv*) síncrono esclavo.

El registro Baud Rate de la USART (UBRRn) en conjunto con un contador descendente (down-counter) funcionan como un prescaler o generador de baud-rate programable. El contador descendente, corriendo a la velocidad de reloj del sistema (f_{osc}), se carga con el valor del registro UBRRn cada vez que alcanza el valor cero o cuando se escribe el registro UBRRnL y genera un pulso de reloj. El reloj del generador de baud-rate tiene entonces una frecuencia $f_{osc}/(UBRRn + 1)$ la cual se divide luego por 2, 8 y 16 dependiendo del modo de operación de la USART.

En el modo asíncrono normal la tasa de transmisión de datos (BR) es

$$BR = \frac{f_{osc}}{16 \cdot (UBRRn + 1)}, \quad (1.1)$$

por lo tanto

$$UBRRn = \frac{f_{osc}}{16 \cdot BR} - 1. \quad (1.2)$$

donde UBRRn es el valor del registro de velocidad.

La tabla 1.2 muestra las velocidades de comunicación más utilizadas y el valor a cargar en el registro UBRRn cuando el μC ATmega328 tiene un oscilador de cristal de cuarzo de 16MHz, como es el caso de la placa Arduino UNO. Se muestra también el error en la velocidad dado el valor del divisor entero.

Baud Rate (bps)	2400	4800	9600	14.4K	19.2K	28.8K	38.4K	57.6K	76.8K	115.2K
UBRRn	416	207	103	68	51	34	25	16	12	8
Error (%)	-0.1	0.2	0.2	0.6	0.2	-0.8	0.2	2.1	0.2	-3.5

Tabla 1.2: Velocidades de comunicación y valor del registro UBRRn para $f_{osc} = 16\text{MHz}$.

Por otro lado, el bloque transmisor consiste en un búfer de escritura, un registro de serialización o desplazamiento (Shift Register), el generador de paridad y la lógica de control para los diferentes formatos de trama; mientras que el receptor consiste de la lógica de control y verificación de paridad, el registro des-serializador y el búfer de recepción de dos niveles. Los errores que pueden producirse en la recepción de datos son: error de trama, error de desbordamiento (OverRun) y error de paridad.

Para poder utilizar la USART en una comunicación es necesario su inicialización. En general, el proceso de inicialización consiste en configurar el baud-rate, el formato de la trama y habilitar el transmisor y/o el receptor. Este proceso se realiza utilizando los registros antes mencionados.

La configuración de la USART0 del μC ATmega328 para una comunicación asíncrona se realiza mediante los siguientes pasos:

1. Configuración del baud-rate: se realiza escribiendo los registros **UBRR0[H:L]**.
2. El formato de la trama de comunicación se configura utilizando el registro **UCSR0C**.
3. La habilitación de la transmisión y recepción se realiza a través del registro **UCSR0B**.

Registros

Para el manejo de la USART0 del μC ATmega328 se utilizan 5 registros: UDR0, UCSROA, UCSROB, UCSROC y UBRR0. Estos registros están mapeados en la memoria RAM del μC (ver figura 1.5), o sea que tienen direcciones de memorias fijas. La figura 1.9 muestra los registros para el manejo de la USART0, los bits que los componen y las direcciones de memoria donde están mapeados. A continuación se describen las funciones de los bits más relevantes.

Registro de datos de la USART0

Los registros búfer de transmisión y de recepción de datos de la USART comparten la misma dirección de entrada/salida como registro UDR0 (USART Data Register). Cuando se realiza una escritura en la dirección del registro UDR0, los datos se almacenan en el registro búfer de transmisión (TXB). Cuando se lee la dirección de memoria del registro UDR0 se obtiene el valor del registro búfer de recepción de datos (RXB). Los bits más altos no son utilizados en el caso de tramas de 5-, 6- y 7-bits cuando se escribe y en la lectura son puestos a cero por el receptor.

El búfer de transmisión solo se puede escribir cuando el bit UDRE0 del registro UCSROA este a 1, en caso contrario los datos escritos serán ignorados por el transmisor de la USART. Cuando los datos se escriben en el búfer de transmisión y este está habilitado, el transmisor cargará el dato en el registro serializador de transmisión (Transmit Shift Register) cuando este este vacío. Luego, los datos se transmitirán en serie por el pin TxD0. En cuanto a la recepción, el búfer de recepción consta de una FIFO de dos niveles, cuyo estado cambia cada vez que se acceda al búfer de recepción.

Registro de control y estado A de la USART0

Las funciones de los bits más utilizados del registro de control y estado A de la USART0 son:

- Bit 7 – RXC0 (USART Receive Complete / Recepción completa): este bit actúa como bandera (flag) para indicar que hay un dato no leído en el búfer de recepción y se borra cuando el búfer de recepción está vacío.

Bit	7	6	5	4	3	2	1	0	
0xC6	RXB[7:0]								UDR0 (Read)
	TXB[7:0]								UDR0 (Write)
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	
0xC0	RXC0	TXC0	UDRE0	FE0	DOR0	UPE0	U2X0	MPCM0	UCSR0A
	R	R/W	R	R	R	R	R/W	R/W	
Read/Write	0	0	1	0	0	0	0	0	
Initial value	0	0	1	0	0	0	0	0	
0xC1	RXCIE0	TXCIE0	UDRIE0	RXEN0	TXEN0	UCSZ02	RXB80	TXB80	UCSR0B
	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Read/Write	0	0	0	0	0	0	0	0	
Initial value	0	0	0	0	0	0	0	0	
0xC2	UMSEL01	UMSEL00	UPM01	UPM00	USBS0	UCSZ01	UCSZ00	UCPOL0	UCSR0C
	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Read/Write	0	0	0	0	0	1	1	0	
Initial value	0	0	0	0	0	1	1	0	
0xC5	-	-	-	-	UBRR0[11:8]				UBRR0H
	UBRR0[7:0]								
0xC4									
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

Figura 1.9: Registros para el manejo de la USART0 del μ C ATmega328.

- Bit 6 – TXC0 (USART Transmit Complete / Transmisión completa): este bit de bandera se pone a 1 cuando la trama completa en el registro de desplazamiento de transmisión ha sido serializado y no hay datos nuevos en el búfer de transmisión (UDR0).
- Bit 5 – UDRE0 (USART Data Register Empty / Registro de datos vacío): este flag indica si el búfer de transmisión (UDR0) está listo para recibir un dato nuevo. Si está a 1 el búfer está vacío y listo para ser escrito.
- Bit 4 – FE0 (Frame Error / Error de trama):
- Bit 3 – DOR0 (Data OverRun / Desbordamiento de datos):
- Bit 2 – UPE0 (USART Parity Error / Error de paridad):

Registro de control y estado B de la USART0

Las funciones de los bits más utilizados del registro de control y estado B de la USART0 son:

- Bit 4 – RXEN0 (Receiver Enable / Receptor habilitado): al escribir un 1 se habilita la recepción de datos y se configura el pin RxD0.
- Bit 3 – TXEN0 (Transmitter Enable / Transmisor habilitado): al escribir un 1 se habilita la transmisión de datos y se configura el pin TxD0.
- Bit 2 – UCSZ02 (Character Size / Tamaño del caracter): este bit, en combinación con los bits UCSZ01:0, configuran la cantidad de bits de datos (Character SiZe) de la trama de transmisión y recepción.

Registro de control y estado C de la USART0

Las funciones de los bits más utilizados del registro de control y estado C de la USART0 son:

- Bits 7:6 – UMSEL01:0 (USART Mode Select / Selección del modo de la USART): estos bits seleccionan el modo de operación de la USART0, donde para el modo asíncrono los valores deben ser 00.

- Bits 5:4 – UPM01:0 (Parity Mode / Modo de paridad): fijan el tipo de paridad utilizada. Estos son: 00, paridad desactivada; 01, reservado; 10, paridad par; y 11, paridad impar.
- Bit 3 – USBS0 (Stop Bit Select / Selección de bit de stop): seleccionan la cantidad de bits de parada agregados en la transmisión (el receptor ignora esta configuración). Las opciones son 0: 1-bit de parada, o 1: 2-bits de parada.
- Bit 2:1 – UCSZ01:0 (Character Size / Tamaño de caracter): estos bits, en conjunto con el bit UCSZ02 en UCSR0B, configuran la cantidad de bits de datos (Character SiZe) utilizada en la trama de transmisión y recepción. La tabla 1.3 muestra las opciones de configuración.

UCSZ02	UCSZ01	UCSZ00	Tamaño de caracter
0	0	0	5 bits
0	0	1	6 bits
0	1	0	7 bits
0	1	1	8 bits
1	0	0	Reservado
1	0	1	Reservado
1	1	0	Reservado
1	1	1	9 bits

Tabla 1.3: Configuración de los bits UCSZ0.

Registros de Baud Rate

Los bits del registro de baud-rate son:

- Bits 15:12: estos bits están reservados para uso futuro.
- Bits 10:0 – UBRR[11:0] (USART Baud Rate): estos 12 bits configuran la velocidad de transmisión (baud-rate) de la USART. El registro UBRR0H contiene los 4 bits más significativos y el registro UBRR0L los 8 bits menos significativos del baud-rate. Al escribir el registro UBRR0L actualiza el prescaler del baud-rate de forma inmediata.

1.4. Programación del ATmega328

El microcontrolador ATmega328 de la placa Arduino UNO puede programarse ya sea utilizando o no el IDE Arduino. Ambas opciones se muestran a continuación en conjunto con sendos ejemplos de programas *Hola mundo* para sistemas embebidos, que consiste en hacer parpadear un LED (Blink) conectado a un pin digital configurado como salida.

Los diferentes archivos de código fuente mostrados como ejemplo a continuación se encuentran en [8]. Si no se dispone de una placa Arduino UNO se puede utilizar el simulador online TinkerCAD® [9].

1.4.1. Programación con el IDE Arduino

Los programas escritos utilizando el Entorno de Desarrollo Integrado o IDE por sus siglas en inglés de Integrated Development Environment se llaman sketch, que podría traducirse como “boceto” [6]. La idea de “sketch” en el contexto de la programación proviene de Processing [10]. Processing es un lenguaje de programación y entorno de desarrollo integrado de código abierto basado en Java para la enseñanza y producción de proyectos multimedia e interactivos, sobre el cual se basó el desarrollo del IDE Arduino. Esta idea se extendió mediante la placa de desarrollo Wiring dentro del contexto de la creación de prototipos de electrónica o “sketching” con hardware [6].

Estudiantes y profesionales han estado utilizando el IDE clásico de Arduino desde que nació Arduino. El IDE 2 de Arduino presenta mejoras sustanciales en relación al IDE clásico, con mayor rendimiento, una interfaz de usuario mejorada y muchas funciones nuevas como el autocompletado de código, un depurador (debugger) integrado y la posibilidad de sincronizar los programas desarrollados con Arduino

Cloud. Arduino IDE 2 incluye además una nueva barra lateral que brinda mayor accesibilidad a las herramientas más útiles.

La ventana o interfaz gráfica de usuario (GUI, Graphical User Interface) del Arduino IDE 2 se muestra en la figura 1.10. Los sketches se escriben en el área del edición de texto del IDE y se guardan con extensión .ino.

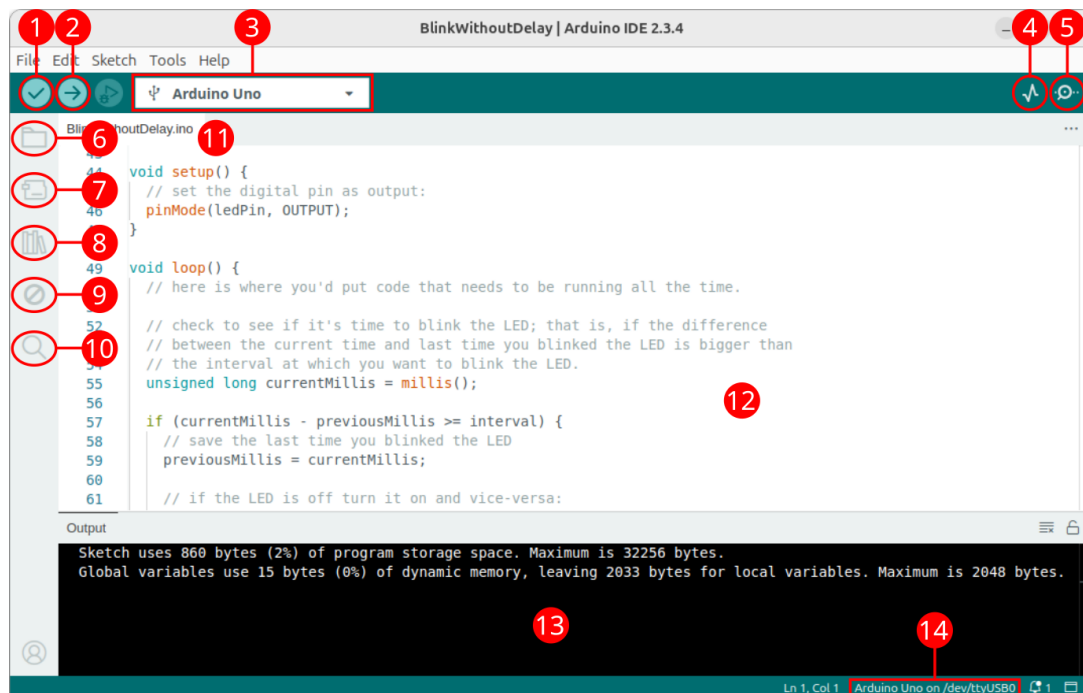


Figura 1.10: Entorno de desarrollo integrado Arduino IDE 2.

Las diferentes partes de la GUI del IDE son:

1. **Construcción (Verify)**: realiza la construcción de los archivos del proyecto.
2. **Grabar (Upload)**: carga el programa construido en la memoria del μ C.
3. **Seleccionar placa y puerto (Select Board & Port)**: muestra las placas Arduino detectadas, junto al número de puerto de conexión.
4. **Graficador serial (Serial Plotter)**: abre la ventana del graficador serial.
5. **Monitor serial (Serial Monitor)**: abre la ventana del monitor serial.
6. **Cuaderno de sketches (Sketchbook)**: listado de todos los sketches almacenados localmente en la computadora.
7. **Administrador de placas (Boards Manager)**: explora los paquetes de Arduino y de terceros que se pueden instalar.
8. **Administrador de bibliotecas (Library Manager)**: explora las bibliotecas de Arduino, creadas por Arduino y su comunidad.
9. **Depurador (Debugger)**: prueba y depura programas en tiempo real.
10. **Búsqueda (Search)**: busca palabras clave en el código fuente.
11. **Nombre del sketch**: muestra el nombre del sketch actual.
12. **Editor**: es el área para la edición del código fuente.
13. **Área de mensajes**: indica si hubo un error o no en la construcción del programa.

14. Modelo de placa y puerto serie: muestra el modelo de placa y puerto serie configurados.

Para utilizar el Arduino IDE 2 en Ubuntu hay que descargar el archivo `arduino-ide_x.y.z_Linux_64bit.zip` [11], donde `x,y,z` hace referencia a la versión del IDE, descomprimir el archivo y ejecutar el binario `arduino-ide`. El IDE de Arduino incluye la biblioteca de programación [12] y el conjunto de herramientas para la construcción de programas (Toolchain) basado en GCC. Para poder utilizar la placa Arduino UNO hay que instalar el paquete **Arduino AVR Boards** desde el "Administrador de placas" (Boards Manager).

Desde el menú **Tools** del IDE se puede seleccionar la placa Arduino particular sobre la que se trabajará con la opción **Board** y el puerto serie por el cual se grabará el programa en la memoria del μC en la opción **Port**.

El monitor serial (Serial Monitor) es una herramienta que incluye el Arduino IDE 2 y actúa como una terminal de comunicación serial para recibir y enviar datos desde y hacia el μC ATmega328. La figura 1.11 muestra el monitor serial, que aparece como panel al lado del área de mensajes del Arduino IDE 2.

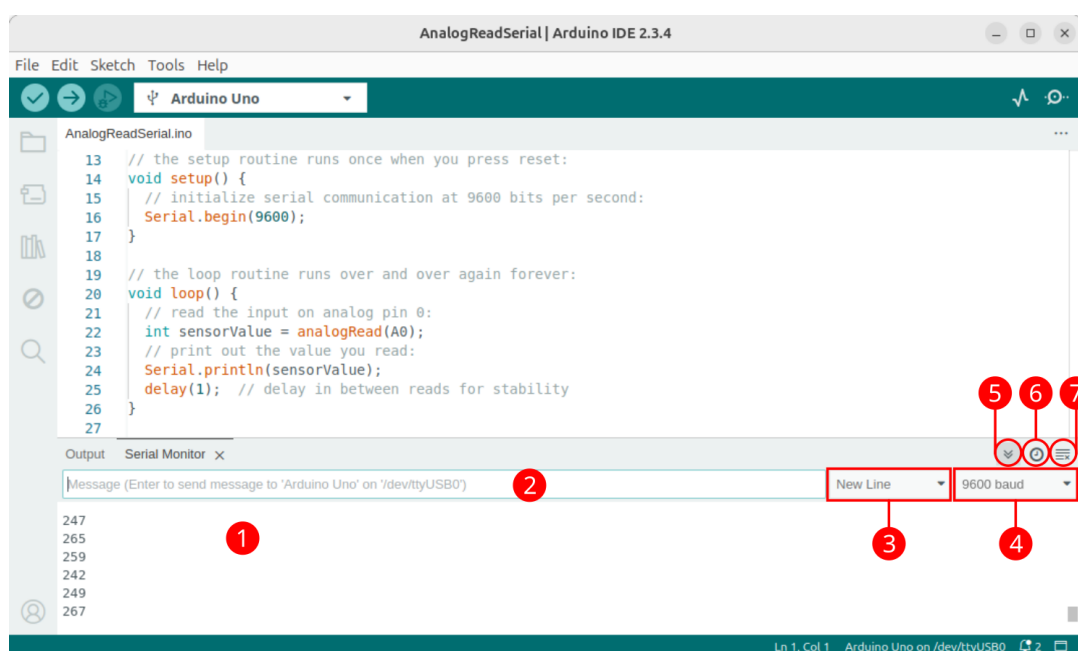


Figura 1.11: Monitor serial del Arduino IDE 2.

Las diferentes partes del monitor serial son:

1. Área donde se muestran los mensajes recibidos desde el μC .
2. Campo de edición para escribir el mensaje a enviar al μC .
3. Menú desplegable para definir el fin del mensaje enviado. Puede ser: No line end (sin final), New line (carácter de nueva línea), Carriage return (carácter de retorno de carro), Both NL & CR (caracteres de nueva línea y retorno de carro).
4. Menú desplegable para seleccionar la velocidad de comunicación (baud).
5. Permite activar/desactivar el desplazamiento (scroll) automática del área de mensajes recibidos.
6. Agrega una marca de tiempo (timestamp) en los mensajes recibidos.
7. Botón para borrar el área de mensajes recibidos.

Programación de sketch

Como se mencionó, los programas escritos con el IDE Arduino se llaman sketches y se guardan con la extensión `.ino`. La programación del sketch consiste en escribir el cuerpo de dos funciones:

- `setup()`: se ejecuta cuando se enciende o reinicia (reset) el microcontrolador.
- `loop()`: se ejecuta de forma constante en un bucle infinito.

En el menú `File->Examples` se encuentran varios sketches de ejemplos que pueden servir como punto de partida para conocer las funciones de la biblioteca Arduino. Algunos de los ejemplos interesantes para estudiar son: `DigitalReadSerial`, `AnalogReadSerial`, `Fade` y `BlinkWithoutDelay`, entre otros.

El listado 1.1 se muestra un sketch para encender y apagar un LED cada 1 seg. En la función `setup()` se configura como salida el pin digital sobre el cual se encuentra conectado el LED llamando a la función `pinMode()`. Luego, en la función `loop()` se enciende y apaga el LED con la función `digitalWrite()` cada un segundo. El retardo de tiempo (`delay`) se realiza con la función bloqueante `delay()` cuyo argumento determina el tiempo de retardo expresado en milisegundos.

Listado 1.1: Código fuente de sketch Arduino `Blink.ino`.

```
1 #define PIN_LED LED_BUILTIN
2
3 // La función 'setup' se ejecuta una única vez al encender la placa
4 // o presionar reset
5 void setup() {
6     // Inicializa el pin digital como salida
7     pinMode(PIN_LED, OUTPUT);
8 }
9
10 // La función 'loop' corre indefinidamente una y otra vez.
11 void loop() {
12     digitalWrite(PIN_LED, HIGH); // Enciende el LED
13     delay(1000);                 // Espera 1 seg.
14     digitalWrite(PIN_LED, LOW);  // Apaga el LED
15     delay(1000);                 // Espera 1 seg.
16 }
```

El sketch del listado 1.1 es similar al ejemplo `Blink` incluido en el IDE. Al ejemplo se le ha agregado la constante simbólica `PIN_LED` para definir el pin sobre el cual está conectado el LED, en este caso el pin dado por `LED_BUILTIN` (pin 13) correspondiente al LED incluido en la placa Arduino UNO.

Dentro del IDE Arduino

En el proceso de construcción del sketch que realiza el IDE Arduino, cada archivo `.ino` se convierte en un archivo con extensión `.cpp`, que luego es compilado con el compilador de GCC en conjunto con los demás archivos fuentes (escritos en lenguaje C y C++) de las bibliotecas incluidas en el IDE. En uno de los archivos que se compila, de nombre `main.cpp`, se encuentra el cuerpo de la función `main()` e incluye además el archivo de cabecera `Arduino.h`.

El listado 1.2 muestra una versión simplificada del archivo `main.cpp`, donde se observan las llamadas a las funciones `setup()` y `loop()`.

El listado 1.3 muestra un fragmento del archivo de cabecera `Arduino.h` de la biblioteca Arduino que se incluye en el archivo `main.cpp`. En este archivo es donde se definen las constantes simbólicas utilizadas para la programación del sketch y se encuentran también los prototipos de funciones de biblioteca como `pinMode()`, `digitalWrite()`, `delay()`, etc.

1.4.2. Programación sin el IDE Arduino

Es posible realizar la construcción y grabación de los programas del microcontrolador ATmega328 de la placa Arduino UNO sin utilizar el IDE Arduino. Para ello es necesario instalar el “Toolchain” para la arquitectura particular del μC , en este caso para arquitectura AVR. El toolchain está basado en el compilador GCC para AVR (`avr-gcc`) el cual en realidad es un compilador cruzado (cross-compiler). Un compilador cruzado es aquel que genera código máquina para una arquitectura diferente a aquella en la cual se ejecuta el compilador.

Listado 1.2: Archivo main.cpp simplificado del IDE Arduino.

```
1 #include <Arduino.h>
2
3 int main(void)
4 {
5     // Llama funciones de inicialización del uC
6
7     setup();
8
9     for (;;) {
10         loop();
11         // Otras operaciones
12     }
13     return 0;
14 }
```

Listado 1.3: Archivo Arduino.h del IDE Arduino.

```
1 #ifndef Arduino_h
2 #define Arduino_h
3
4 #define HIGH 0x1
5 #define LOW 0x0
6
7 #define INPUT 0x0
8 #define OUTPUT 0x1
9
10 void pinMode(uint8_t, uint8_t);
11 void digitalWrite(uint8_t, uint8_t);
12 int digitalRead(uint8_t);
13 int analogRead(uint8_t);
14 void analogWrite(uint8_t, int);
15
16 unsigned long millis(void);
17 unsigned long micros(void);
18 void delay(unsigned long);
19 void delayMicroseconds(unsigned int us);
20
21 #endif
```

El Toolchain de GCC para AVR

El Toolchain de GCC para arquitectura AVR está compuesto por:

- **gcc**: el compilador de C y C++ [13].
- **binutils**: colección de herramientas para la manipulación de archivos binarios.
- **libc-avr**: subconjunto de la biblioteca estándar de C con funciones específicas para arquitectura AVR [14].
- **avrdude**: programa para escribir y leer la memoria Flash del μ C.

La instalación del Toolchain en Ubuntu se realiza instalando los siguientes paquetes: **gcc-avr**, **binutils-avr**, **avr-libc** y **avrdude**.

El programa del listado 1.4 hace encender y apagar un LED cada 1 segundo conectado a una salida digital, en este caso el bit 5 del Puerto B que, como puede verse en la figura 1.2, corresponde al LED incluido en la placa Arduino UNO. Antes del bucle infinito se configura este bit en particular como salida y en el bucle infinito se escribe un 1 (uno) y un 0 (cero) lógico cada 1 segundo.

Para construir el programa y grabarlo a la memoria del μ C ATmega328 de la placa Arduino UNO hay que realizar los siguientes pasos:

1. Construcción del código fuente (con extensión .c) hasta la etapa de ensamblado, de la cual se obtiene un archivo objeto (con extensión .o):
`avr-gcc -Os -DF_CPU=16000000UL -mmcu=atmega328p -c -o blink.o blink.c`
2. Enlazado con bibliotecas, del cual se obtiene un archivo binario (extensión .elf):
`avr-gcc -mmcu=atmega328p blink.o -o blink.elf`

Listado 1.4: Código fuente en lenguaje C, `blink.c`.

```

1 #include <avr/io.h>
2 #include <util/delay.h>
3
4 #define BLINK_DELAY_MS 1000
5
6 int main (void)
7 {
8     /* Configura el bit 5 del Puerto B como salida */
9     DDRB |= _BV(DDRB5);
10
11     while(1)
12     {
13         /* Pone el bit 5 en alto (enciende el LED) */
14         PORTB |= _BV(PORTB5);
15         _delay_ms(BLINK_DELAY_MS);
16
17         /* Pone el bit 5 en bajo (apaga el LED) */
18         PORTB &= ~_BV(PORTB5);
19         _delay_ms(BLINK_DELAY_MS);
20     }
21     return 0;
22 }

```

3. Conversión del código binario (`.elf`) a formato objeto hexadecimal Intel (`.hex`):
`avr-objcopy -O ihex -R .eeprom blink.elf blink.hex`
4. Grabación en la memoria de programa del μ C:
`avrdude -F -V -c arduino -p ATMEGA328P -P /dev/ttyACM0 -b 115200 -U flash:w:blink.hex`

En (1) se realiza la construcción hasta la etapa de ensamblado (preprocesado, compilado y ensamblado) para convertir el archivo fuente con extensión `.c` al archivo objeto con extensión `.o`, donde: *i*) la bandera (flag) `-c` le indica que detenga la construcción antes de la etapa de enlazado, *ii*) la opción de optimización `-Os` realiza una optimización en el tamaño (size) del binario, *iii*) se define la constante simbólica `F_CPU` a `16000000UL` para la función de retardo y *iv*) define el procesador con la opción `-mmcu` a `atmega328p` [15]. En (2) se enlaza el archivo objeto con extensión `.o` y se obtiene el archivo binario con extensión `.elf` (Executable and Linkable Format). En esta etapa también se enlaza el archivo objeto donde está el código de inicio (startup) del μ C, el cual realiza diferentes funciones de inicialización y llama a la función `main()` escrita en lenguaje C. En (3) se convierte el archivo binario del formato ELF con extensión `.elf` a formato HEX de Intel (opción `-O`) con extensión `.hex`. En (4) se graba el archivo HEX a la memoria Flash del μ C ATmega328 (opción `-p`) utilizando como programador la placa Arduino (opción `-c`) a través del puerto serie (opción `-P`) `/dev/ttyACM0` a una velocidad de comunicación (opción `-b`) de 115200 bits por segundo.

Cabe aclarar que el código fuente del listado 1.4 puede ser transcrito dentro de un sketch utilizando el IDE Arduino. Para ello hay que colocar las líneas de código antes del bucle infinito dentro de la función `setup()` del sketch y lo que está dentro del bucle infinito en la función `loop()`. No es necesario incluir los archivos de cabecera, dado que ya son incluidos por la biblioteca del IDE Arduino. Se puede utilizar entonces el IDE Arduino para construir el programa y grabarlo a la memoria del μ C.

1.5. Programación de los puertos digitales

1.5.1. Programación de salidas digitales

El código fuente del listado 1.4 es un ejemplo de manejo de un puerto digital de entrada/salida. En este caso se utiliza el bit 5 del puerto B configurado como salida para encender y apagar un LED conectado al pin correspondiente. La placa Arduino UNO incluye un LED conectado a dicho pin (ver figura 1.2).

En la línea 9 del programa se configura el bit 5 del puerto B como salida modificando el valor del registro `DDRB`. Se utiliza el operador OR a nivel de bits para poner a 1 lógico dicho bit y así configurarlo como salida, como se describió en la sección 1.3.2. Dicha línea es equivalente a:

```
DDRB = DDRB | _BV(DDB5);
```

La macro `_BV()` (Bit Value) toma el valor del bit y lo convierte en un byte con dicho bit puesto a 1 y los demás a 0. Esta macro está definida como:

```
#define _BV(bit) (1 << (bit))
```

y `DDB5` es una constante simbólica de valor 5. Por lo que la operación queda:

```
DDRB = DDRB | (1 << (5)); /* DDRB = DDRB | 0x20 */
```

con lo cual del registro `DDRB` tendrá el mismo valor que antes con el bit 5 puesto a 1.

Luego, en las líneas 14 y 19 se realiza una operación similar para poner a 1 y 0 lógico respectivamente el bit 5 del registro `PORTB` que, como se mencionó (sección 1.3.2), actúa directamente sobre el pin digital correspondiente, en este caso `PB5`. En la línea 14 se utiliza el operador OR lógico a nivel de bits para forzar a 1 un bit, mientras que en la línea 19 se utiliza el operador AND y NOT a nivel de bit para forzar a 0. En estas líneas se utiliza también la constante simbólica `PORTB5` que vale 5. La operación que se realiza en la línea 19 es por lo tanto

```
PORTB = PORTB & ~(1 << (5)); /* PORTB = PORTB & 0xDF */
```

El programa del listado 1.5 muestra otra forma de hacer parpadear un LED utilizando el operador XOR (OR exclusivo).

Listado 1.5: Código fuente en lenguaje C, `blink_xor.c`.

```
1 #include <avr/io.h>
2 #include <util/delay.h>
3
4 #define BLINK_DELAY_MS 1000
5
6 int main(void)
7 {
8     /* Configura el bit 5 del Puerto B como salida */
9     DDRB |= _BV(DDB5);
10
11     while(1)
12     {
13         PORTB ^= _BV(PORTB5); /* Toggle LED */
14         _delay_ms(BLINK_DELAY_MS);
15     }
16     return 0;
17 }
```

La figura 1.12 muestra un circuito utilizando la placa Arduino UNO a la que se le conectan 4 LEDs en los bits menos significativos del Puerto D, o sea en los pines `PD0` a `PD3`. El listado 1.6 muestra el código fuente de un programa que muestra sobre los LEDs una cuenta binaria de 4 bits, o sea del rango de valores que va desde `0d = 0x0 = 0000b` hasta `15d = 0xF = 1111b`.

1.5.2. Programación de entradas digitales

La figura 1.13 muestra un circuito utilizando la placa Arduino UNO a la cual se le conecta un pulsador (switch) y un LED. El pulsador se conecta al bit 7 del puerto D (`PD7`) el cual debe configurarse como entrada y el LED se conecta al bit 1 del puerto B (`PB1`) que debe configurarse como salida (ver figura 1.2). El circuito utilizado es parte de la placa “ponchitoCIIP” [3].

El programa del listado 1.7 maneja los puertos digitales del μC para realizar la siguiente función: leer el valor de la entrada digital conectada al pulsador y en caso de estar presionado encender el LED durante 1 seg. y luego apagarlo.

El programa comienza definiendo algunas constantes simbólicas de máscara y estado del pulsador. Luego, en la función `main()` se inicializan los puertos digitales de entrada/salida, configurando el bit del pulsador como entrada y el del LED como salida. Finalmente, en el bucle infinito se lee el valor de la entrada conectada al pulsador y en caso de estar en estado de presionado se actúa sobre la salida conectada al LED.

La constante simbólica `SW_MASK` se utiliza como máscara para la lectura de la entrada digital (línea 23). La línea que realiza la lectura de la entrada es:

```
sw = (PIND & SW_MASK) >> PIND7;
```

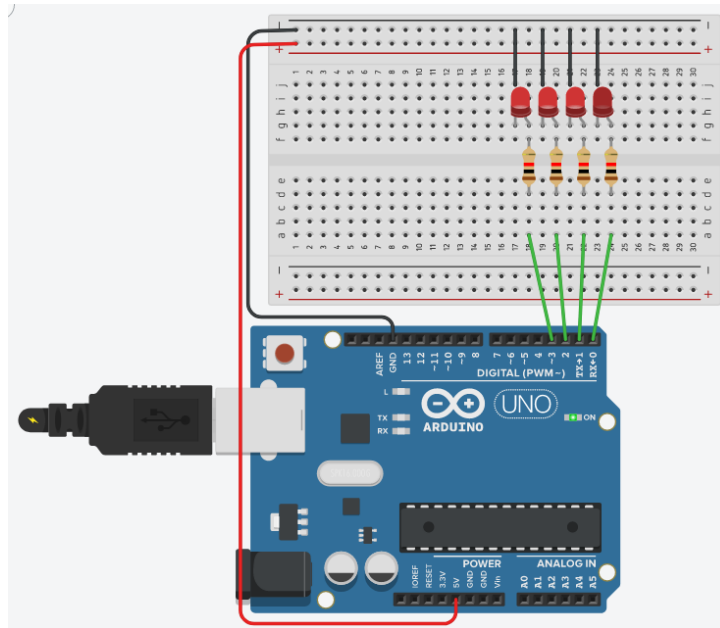


Figura 1.12: Circuito para el programa `led_hex_couter`.

Listado 1.6: Código fuente en lenguaje C, `led_hex_couter.c`.

```

1 #include <avr/io.h>
2 #include <util/delay.h>
3
4 #define LED_MASK 0x0F
5 #define DELAY_MS 500
6
7 int main (void)
8 {
9     unsigned char hex = 0;
10
11     /* Inicializa salidas digitales */
12     DDRD |= LED_MASK;
13
14     while(1)
15     {
16         PORTD &= ~(LED_MASK); // Apaga los LEDs
17         PORTD |= hex;         // Muestra valor
18
19         if(++hex > 16)        // Controla rango máximo
20             hex = 0;
21
22         _delay_ms(DELAY_MS);
23     }
24     return 0;
25 }

```

donde se utiliza el valor del registro `PIND` para la lectura del Puerto D, la que se enmascara con `SW_MASK`, o sea que se ponen a cero todos los bits menos aquel cuyo valor interesa (a esto se le denomina máscara). Este valor luego se desplaza hacia la derecha para ocupar el bit menos significativo de la variable de 8 bits de nombre `sw`. Luego, se utiliza esta variable en la estructura de selección `if` para determinar si hay que encender el LED. La constante simbólica `SW_PRES` fija el valor que tendrá la entrada digital si el pulsador se encuentra presionado, lo cual dependerá de cómo este armado el circuito.

1.5.3. Constantes simbólicas y macros

En esta sección se muestran algunos detalles de la biblioteca de programación de los μC AVR.

El archivo de cabecera `avr/io.h` incluye, mediante compilación condicional (usando la directiva de preprocesador `#if defined`), el archivo de cabecera particular del μC utilizado, en este caso `avr/iom328p.h`. Este archivo contiene diferentes constantes simbólicas y macros para acceder a los registros del μC . El

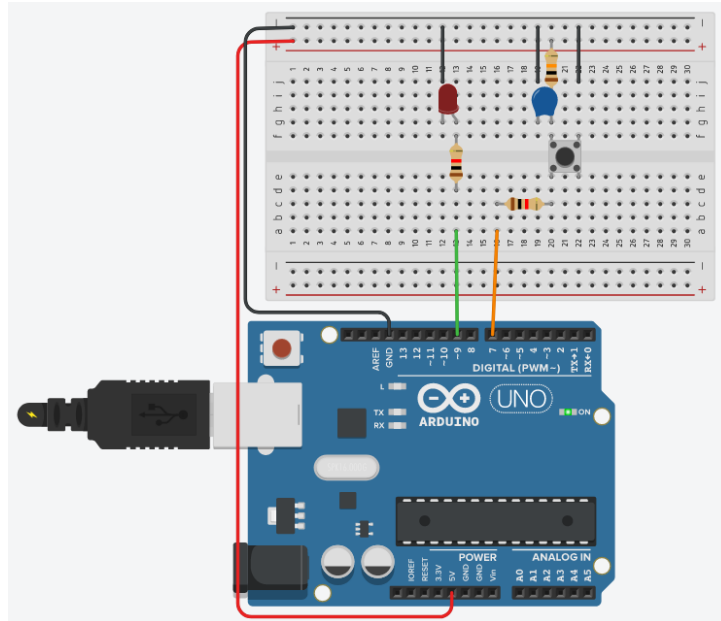


Figura 1.13: Circuito para el programa led_sw.

Listado 1.7: Código fuente en lenguaje C, led_sw.c.

```

1 #include <avr/io.h>
2 #include <util/delay.h>
3
4 /*
5  * LED: pin 9 --> PB1 (Salida)
6  * Sw: pin 7 --> PD7 (Entarda)
7  */
8 #define SW_MASK _BV(PIND7)
9 #define SW_PRES 0
10 #define SW_NO_PRES 1
11
12 int main(void)
13 {
14     unsigned char sw = SW_NO_PRES;
15
16     /* Configuración de entrada/salida digitales */
17     DDRB |= _BV(DDB1); /* LED como salida */
18     DDRD &= ~_BV(DDD7); /* Sw como entrada */
19     PORTB &= ~_BV(PORTB1); /* Apaga el LED */
20
21     while(1)
22     {
23         /* Lee el valor del pulsador/Sw */
24         sw = (PIND & SW_MASK) >> PIND7;
25
26         if(sw == SW_PRES)
27         {
28             PORTB |= _BV(PORTB1); /* Enciende el LED */
29             _delay_ms(1000);
30             PORTB &= ~_BV(PORTB1); /* Apaga el LED */
31         }
32     }
33     return 0;
34 }

```

listado 1.8 muestra una sección de este archivo de cabecera.

Como se observa, se definen los bits de los registros, tales como PINB1, DDR5, PORTB0, etc.; y los registros en sí mismo, tales como DDRB y PORTB.

La macro `_SFR_IO8()` está definida en el archivo de cabecera `avr/sfr_defs.h` (Special Function Registers macros) como:

```
#define _SFR_IO8(io_addr) _MMIO_BYTE((io_addr) + __SFR_OFFSET)
```

Listado 1.8: Sección del archivo de cabecera iom328p.h.

```

1 #define PINB _SFR_I08(0x03)
2 #define PINB0 0
3 #define PINB1 1
4 #define PINB2 2
5
6 #define DDRB _SFR_I08(0x04)
7 #define DDB0 0
8 #define DDB1 1
9 #define DDB2 2
10
11 #define PORTB _SFR_I08(0x05)
12 #define PORTB0 0
13 #define PORTB1 1
14 #define PORTB2 2

```

donde

```
#define _MMIO_BYTE(mem_addr) (*(volatile uint8_t *) (mem_addr))
```

y `__SFR_OFFSET` es una constante simbólica con la dirección de memoria base de los registros de funciones especiales. Por lo tanto, la macro `_MMIO_Byte()` se sustituye como un puntero desreferenciado a la dirección de memoria de un registro en particular.

Por ejemplo, al hacer referencia al registro `DDRB` la macro se expande a

```
((*(volatile uint8_t *) ((0x04) + 0x20))
```

o sea, un puntero desreferenciado a la dirección `0x24` (ver tabla 1.1). Acá puede observarse el uso del calificador `volatile` del lenguaje C. El calificador `volatile` le indica al compilador no optimizar lo relacionado a dicha variable. Este calificador se utiliza principalmente para el acceso a hardware como en este caso.

1.6. Programación de la UART

Como se mencionó anteriormente, para poder utilizar la USART en una comunicación entre el μ C ATmega328 y otro dispositivo es necesario su inicialización, lo que consiste en configurar el formato de la trama y la velocidad de comunicación, además de habilitar el transmisor y/o receptor dependiendo de cada caso. En las secciones siguientes se muestra cómo configurar la USART0 del μ C ATmega328 para el modo asíncrono (UART), transmitir y recibir datos hacia y desde la PC. En el lado de la PC se puede utilizar ya sea el monitor serial del IDE Arduino o bien algún otro programa de terminal serial.

1.6.1. Configuración de la USART

La configuración de la USART consiste en seleccionar el modo de funcionamiento asíncrono (UART), definir el tipo de trama y la velocidad de comunicación. El tipo de trama queda determinada por la cantidad de bits de datos, la cantidad de bits de parada y si se habilita o no la detección de errores mediante paridad y el tipo de paridad (par o impar) a utilizar, como se describió en la sección 1.3.3. La velocidad de comunicación se refiere a la cantidad de bits por segundos (bps) o baud-rates que son serializados en la transmisión/recepción de la trama. El formato de trama suele indicarse por una combinación de dos números y una letra donde:

- El primer número indica la cantidad de bits de datos.
- La letra indica el tipo de paridad, donde: **N** señala que no se utiliza el control de paridad, **E** (even) para la paridad par y **O** (odd) para impar.
- El último número indica la cantidad de bits de stop.

Un formato de trama muy utilizado es `8N1`, o sea que la trama está conformada por 8-bits de datos, sin control de paridad y con 1-bit de parada. Otros ejemplos son: `5E2`, `7E1`, `6O1`, etc.

Para la configuración de la USART0 del μ C ATmega328 se utilizan los registros de control y estado `B` y `C`, `UCSROB` y `UCSROC` (ver sección 1.3.3).

El código fuente del listado 1.9 es un ejemplo de manejo de la USART0 del μ C ATmega328. En este programa se configura la USART0 en modo asíncrono con una trama 8N1 y a una velocidad de 115.200bps. Según la tabla 1.2, para lograr esta velocidad de comunicación hay que escribir el valor 8 al registro UBRR0. El programa define una variable `ubrr` de 16-bits cuyo valor de inicialización está dado por la constante simbólica `BAUD_RATE_115200`. Luego, en la sección de configuración (antes del bucle infinito), se configura la velocidad mediante las líneas:

```
UBRR0L = (uint8_t)(ubrr & 0x00FF);
UBRR0H = (uint8_t)(ubrr >> 8);
```

donde en la primer línea se obtiene los 8-bits menos significativos (LSB, Least Significant Byte) con una máscara y se carga en el registro UBRR0L (byte menos significativo del registro UBRR); y en la segunda línea se obtienen los 8-bits más significativos (MSB, Most Significant Byte) de la variable de 16-bits mediante una operación de desplazamiento hacia la derecha, y el valor resultante se carga en el registro UBRR0H (byte más significativo del registro UBRR).

Las mismas operaciones puede utilizarse para otras velocidades de comunicación definidas por otras constantes simbólicas tales como:

```
#define BAUD_RATE_4800    207
#define BAUD_RATE_9600    103
#define BAUD_RATE_38400   25
#define BAUD_RATE_57600   16
```

Luego se configura la trama al formato 8N1, fijando la cantidad de bits a 8:

```
UCSR0C |= _BV(UCSZ00) | _BV(UCSZ01);
UCSR0B &= ~_BV(UCSZ02);
```

sin control de paridad y 1 bit de parada:

```
UCSR0C &= ~( _BV(UPM01) | _BV(UPM00) );
UCSR0C &= ~_BV(USBS0);
```

Aún cuando hay valores por defecto de los registros (ver figura 1.9), al configurarlos de forma explícita el programa queda mejor auto-documentado.

1.6.2. Transmisión de datos

Una vez configurada la velocidad de transmisión y el formato de la trama de comunicación, para que el μ C ATmega328 pueda enviar datos por la USART0 es necesario habilitar el transmisor. El transmisor se habilita escribiendo un 1 en el bit `TXEN0` del registro `UCSR0B`. Esto termina de configurar la USART0 para la transmisión, como se muestra en el programa del listado 1.9.

Este programa envía por puerto serie la cadena **Hola mundo** (incluido el caracter de nueva línea `'\n'` y retorno de carro `'\r'`). La transmisión de la cadena se realiza en el bucle principal del programa que se ejecuta cada 500ms utilizando un bucle `for`.

Para poder transmitir un dato por la USART hay que escribir el registro de transmisión de datos, `UDR0` (ver sección 1.3.3), habiendo verificado previamente que el búfer de transmisión está vacío. El estado del búfer de transmisión se verifica según el valor del bit `UDRE0` del registro `UCSR0A`. La línea:

```
while( !(UCSR0A & _BV(UDRE0)) );
```

espera a que el búfer de transmisión este vacío. El estado de búfer se obtiene con la operación `(UCSR0A & _BV(UDRE0))` que devuelve 1 si el búfer está vacío. El bucle `while`, el cual no contiene sentencia (observar el punto y coma al final de la línea) se ejecuta mientras el búfer no este vacío. Cuando el búfer este vacío, la línea:

```
UDR0 = dato[i];
```

escribe el siguiente caracter de la cadena a enviar en el registro de transmisión de datos, `UDR0`.

La figura 1.14 muestra la aplicación `cutecom` donde se observa la cadena recibida por la PC desde el μ C ATmega328.

1.6.3. Recepción de datos

La figura 1.15 muestra un circuito utilizando la placa Arduino UNO a la cual se le conecta un LED rojo y otro verde a las salidas digitales PB1 y PD5, respectivamente. El LED rojo cambiará de estado

Listado 1.9: Código fuente en lenguaje C, `hola_mundo.c`.

```

1 #include <avr/io.h>
2 #include <util/delay.h>
3 #include <stdint.h>
4
5 #define BAUD_RATE_115200 8
6
7 int main()
8 {
9     int i;
10    uint16_t ubrr = BAUD_RATE_115200;
11    uint8_t dato[] = "Hola_mundo\n\r";
12
13    /* Configura modo asíncrono (UART) */
14    UCSRC &= ~( _BV(UMSEL01) | _BV(UMSEL00) );
15
16    /* Configura el baud-rate */
17    UBRR0L = (uint8_t)(ubrr & 0x00FF); /* Byte menos significativo */
18    UBRR0H = (uint8_t)(ubrr >> 8);      /* Byte más significativo */
19
20    /* Configura la trama con 8 bits de datos */
21    UCSRC |= _BV(UCSZ00) | _BV(UCSZ01);
22    UCSRB &= ~_BV(UCSZ02);
23
24    /* Paridad desactivada (N) y 1 bit de stop */
25    UCSRC &= ~( _BV(UPM01) | _BV(UPM00) );
26    UCSRC &= ~_BV(USBS0);
27
28    /* Habilita el transmisor */
29    UCSRB |= _BV(TXEN0);
30
31    while(1)
32    {
33        for(i = 0; dato[i] != '\0'; i++)
34        {
35            /* Espera a que el búfer de transmisión esté vacío */
36            while( !(UCSRA & _BV(UDRE0)) )
37            ;
38            UDR0 = dato[i]; /* Escribe el búfer de transmisión */
39        }
40        _delay_ms(500);
41    }
42    return 0;
43 }

```

(encendido a apagado y apagado a encendido) cuando el μ C reciba el caracter 'r' o 'R'. Lo mismo para el LED verde al recibir el caracter 'v' o 'V'. El circuito utilizado es parte de la placa “ponchitoCIIT” [3].

El programa del listado 1.10 maneja la USART del μ C para realizar la siguiente función: configura la USART0 en modo asíncrono (UART) con una velocidad de comunicación de 115.200bps y trama 8N1, habilita la recepción y actúa sobre los LEDs conectado a salidas digitales dependiendo del caracter recibido por la UART. La configuración de la USART0 se realiza como se describió en la sección 1.6.1 con la única diferencia de que en lugar de habilitar la transmisión de datos se habilita la recepción. Esto se realiza escribiendo un 1 en el bit `RXEN0` del registro `UCSR0B`. Además, se configuran como salida los pines digitales donde se encuentran conectados los LEDs.

En el bucle principal del programa se espera hasta recibir un caracter con la línea:

```
while( !(UCSRA & _BV(RXC0)) );
```

de igual forma que cuando se espera que el búfer de transmisión este vacío para la transmisión de datos. Luego se utiliza la estructura de selección `switch-case` para actuar sobre el LED adecuado dependiendo del caracter recibido.

El programa del listado 1.11 configura la UART y habilita tanto la recepción como la transmisión de datos. Luego, espera un caracter y lo reenvía salvo que el caracter sea una letra en minúsculas (de 'a' a 'z'), en este caso, la letra se convierte a mayúsculas antes de reenviarla. La figura 1.16 muestra la ventana de `cuteocom` donde se observan las cadenas enviadas y recibidas hacia y desde el μ C ATmega328 convertidas a mayúsculas.

La conversión de caracteres de minúsculas a mayúsculas puede realizarse también utilizando funciones de la biblioteca estándar de C, como:

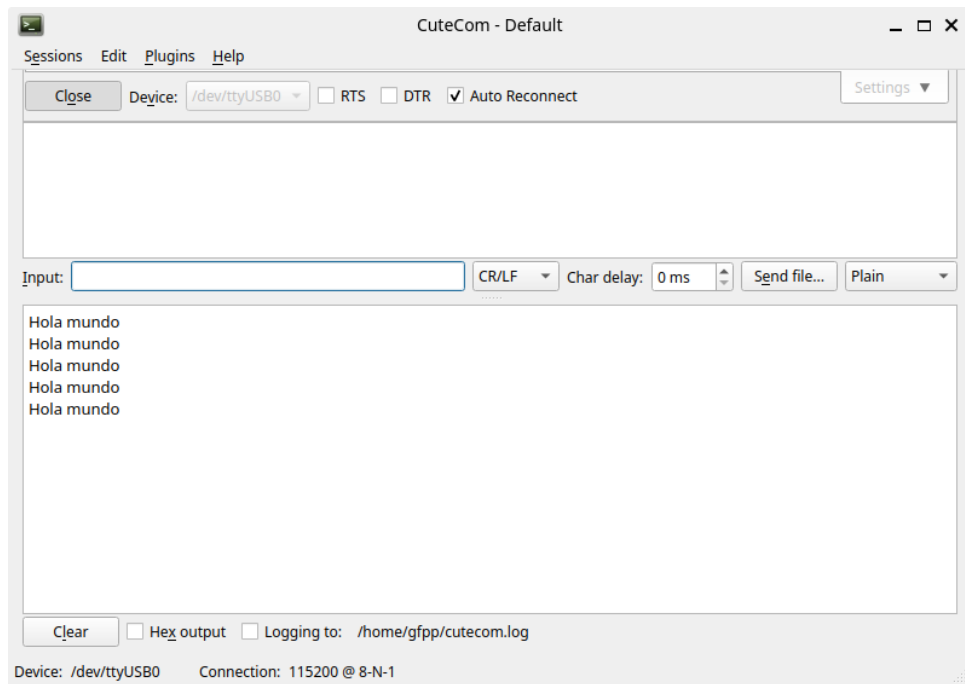


Figura 1.14: Aplicación cutecom con mensaje recibido.

```
if( isalpha(car) )
    car = toupper(car);
```

para lo cual es necesario incluir el archivo de cabecera `ctype.h`.

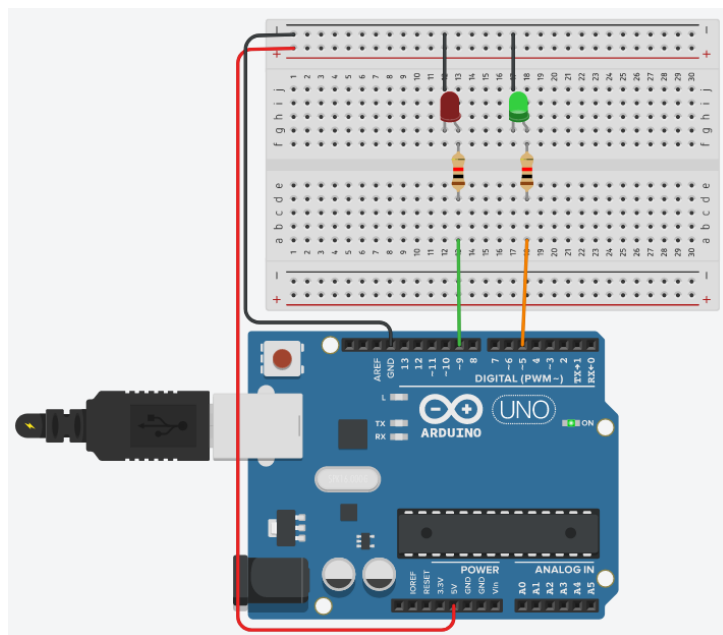


Figura 1.15: Circuito para el programa `uart_read`.

Listado 1.10: Código fuente en lenguaje C, `uart_read.c`.

```
1 #include <avr/io.h>
2 #include <stdint.h>
3
4 #define BAUD_RATE_115200 8
5
6 int main()
7 {
8     uint16_t ubrr = BAUD_RATE_115200;
9
10    DDRB |= _BV(DDB1);          /* Led rojo, salida */
11    DDRD |= _BV(DDD5);          /* Led verde, salida */
12
13    /* Configura modo asíncrono (UART) */
14    UCSROC &= ~( _BV(UMSEL01) | _BV(UMSEL00) );
15
16    /* Configura el baud-rate */
17    UBRROL = (uint8_t)(ubrr & 0x00FF); /* Byte menos significativo */
18    UBRR0H = (uint8_t)(ubrr >> 8);     /* Byte más significativo */
19
20    /* Configura la trama con 8 bits de datos */
21    UCSROC |= _BV(UCSZ00) | _BV(UCSZ01);
22    UCSROB &= ~_BV(UCSZ02);
23
24    /* Paridad desactivada (N) y 1 bit de stop */
25    UCSROC &= ~( _BV(UPM01) | _BV(UPM00) );
26    UCSROC &= ~_BV(USBS0);
27
28    /* Habilita el receptor */
29    UCSROB |= _BV(RXEN0);
30
31    while(1)
32    {
33        /* Espera a recibir un dato */
34        while( !(UCSROA & _BV(RXC0)) )
35            ;
36
37        switch(UDRO) {
38            case 'r':
39                PORTB ^= _BV(PORTB1); /* Cambia estado de LED rojo */
40                break;
41
42            case 'v':
43            case 'V':
44                PORTD ^= _BV(PORTD5); /* Cambia estado de LED verde */
45                break;
46        }
47    }
48    return 0;
49 }
50 }
```

Listado 1.11: Código fuente en lenguaje C, `uart_read_write.c`.

```
1 #include <avr/io.h>
2 #include <stdint.h>
3
4 #define BAUD_RATE_115200 8
5
6 int main()
7 {
8     uint16_t ubrr = BAUD_RATE_115200;
9
10    DDRB |= _BV(DDB1);          /* Led rojo, salida */
11    DDRD |= _BV(DDD5);          /* Led verde, salida */
12
13    /* Configura modo asíncrono (UART) */
14    UCSROC &= ~( _BV(UMSEL01) | _BV(UMSEL00) );
15
16    /* Configura el baud-rate */
17    UBRROL = (uint8_t)(ubrr & 0x00FF); /* Byte menos significativo */
18    UBRR0H = (uint8_t)(ubrr >> 8);     /* Byte más significativo */
19
20    /* Configura la trama con 8 bits de datos */
21    UCSROC |= _BV(UCSZ00) | _BV(UCSZ01);
22    UCSROB &= ~_BV(UCSZ02);
23
24    /* Paridad desactivada (N) y 1 bit de stop */
25    UCSROC &= ~( _BV(UPM01) | _BV(UPM00) );
26    UCSROC &= ~_BV(USBS0);
27
28    /* Habilita el receptor */
29    UCSROB |= _BV(RXEN0);
30
31    while(1)
32    {
33        /* Espera a recibir un dato */
34        while( !(UCSROA & _BV(RXC0)) );
35
36        switch(UDRO) {
37            case 'r':
38            case 'R':
39                PORTB ^= _BV(PORTB1); /* Cambia estado de LED rojo */
40                break;
41
42            case 'v':
43            case 'V':
44                PORTD ^= _BV(PORTD5); /* Cambia estado de LED verde */
45                break;
46        }
47    }
48    return 0;
49 }
```

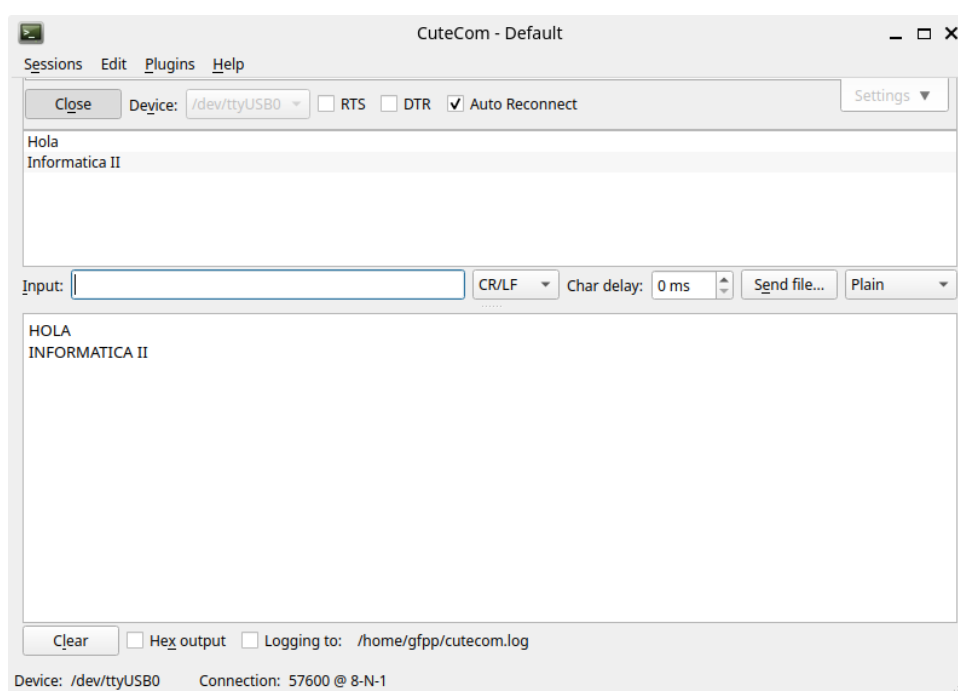


Figura 1.16: Ejemplo del programa `uart_read_write`.

Capítulo 2

Comunicación serie

2.1. Introducción

En informática, la comunicación o transmisión de datos entre dispositivos se puede realizar de forma paralela o serie. La comunicación paralela o en paralelo permite transmitir varios bits (del inglés *binary digit* o dígito binario) de forma simultanea. En contrapartida, la comunicación serie o serial transmite un único bit a la vez de forma secuencial. La principal diferencia entre un canal de comunicación paralelo y uno serie es la cantidad de conductores eléctricos (cables) en la capa física para la transmisión de los bits; por ejemplo, en la comunicación en paralelo se necesitan tantos conductores como bits se desea transmitir.

2.2. Comunicación paralela y serie

La transmisión de datos permite comunicar equipos electrónicos tales como PC, PLC (Controladores Lógicos Programables), instrumentos de laboratorios (multímetros, osciloscopios, etc.), placas de desarrollo de sistemas embebidos (Arduino, Raspberry Pi, etc.); como así también entre componentes internos de una PC.

Algunos ejemplos de sistemas de comunicación paralela son las interfaces (buses) que permiten interconectar componentes internos de una PC tales como:

- ISA: Industry Standard Architecture.
- ATA/IDE: Advanced Technology Attachment/Integrated Drive Electronics.
- SCSI: Small Computer System Interface.
- PCI: Peripheral Component Interconnect.

En cuanto la comunicación de la PC con dispositivos externos se pueden mencionar:

- IEEE-1284 (Standard Signaling Method for a Bi-directional Parallel Peripheral Interface for Personal Computers): incluido en la primeras PC de IBM y luego estandarizado por el Institute of Electrical and Electronics Engineers (IEEE). Brinda una comunicación de “alta velocidad” y bidireccional entre una PC y un dispositivo externo.
- IEEE-488: originalmente desarrollados por Hewlett-Packard (HP-IB: Hewlett-Packard Instrument Bus o GPIB: General-Purpose Instrumentation Bus) para conectar dispositivos de testeo y medición (por ejemplo multímetros, osciloscopios, etc.) con una PC. Luego estandarizado por el IEEE.

Por el lado de la comunicación serie se pueden mencionar los estándares RS-232, RS-422, RS-485 y protocolos como el SPI, I²C, 1-Wire, entre otros.

- RS-232/EIA-232 (Recommended Standard/Electronic Industries Alliance): define las señales que se conectan entre un DTE (Data Terminal Equipment o Equipo Terminal de Datos) como un terminal de computadora y un DCE (Data Communication Equipment o Equipo de Comunicación de Datos) como un módem.

- RS-485/EIA-485: estándar de comunicación en bus diferencial de la capa física del Modelo OSI (Open System Interconnection).

Otras interfaces de comunicación serial son:

- SPI (Serial Peripheral Interface): especificación de comunicación serie usado principalmente para la transferencia de información entre circuitos integrados en equipos electrónicos.
- I²C (Inter-Integrated Circuit) o TWI (Two Wire Interface): bus serie de datos desarrollado por Philips Semiconductors (hoy NXP Semiconductors, parte de Qualcomm), utilizado principalmente para la comunicación entre diferentes partes de un circuito, por ejemplo, entre un controlador y circuitos periféricos integrados.
- 1-Wire: protocolo de comunicaciones diseñado por Dallas Semiconductor basado en un bus, un maestro y varios esclavos de una sola línea de datos.

La figura 2.1 muestra una representación esquemática de dispositivos conectados en paralelo (figura 2.1a) y en serie (figura 2.1b). Como se observa, en la comunicación paralela se necesitan tantas conexiones como bits se quieran transmitir, mientras que para una comunicación serie en un solo sentido se necesitan solo una conexión. Cabe aclarar que esta es una representación esquemática y no un diagrama eléctrico en cuyo caso sería necesario agregar también la señal de referencia de tensión (GND) entre ambos dispositivos.

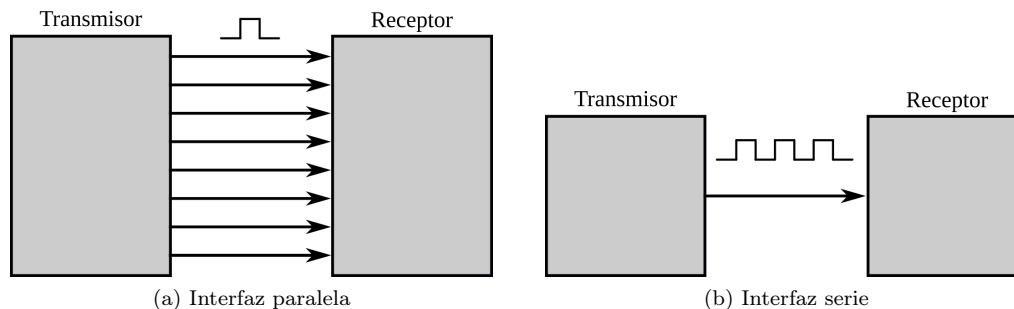


Figura 2.1: Comunicación paralela vs. serie.

Una posible aplicación de comunicación en paralelo entre dos dispositivos en un sistema embebido es la conexión entre una placa de desarrollo como Arduino y una pantalla LCD (Liquid-Crystal Display) ya sea de caracteres o gráfica. Por otro lado, en los sistemas embebidos la transmisión de datos en serie se utiliza en general en módulos de comunicación inalámbrica Bluetooth, módulos GPS (Global Positioning System), GSM/GPRS (Global System for Mobile communications / General Packet Radio Services), etc.

La tabla 2.1 muestra la comparación entre algunas de las características de la comunicación serie y paralela.

2.3. Transmisión síncrona vs. asíncrona

Cuando dos dispositivos se comunican, intercambian algún tipo de señal detectable que representa los datos y a menos que los dispositivos estén tan ocupados que nunca dejen de hablar, habrá un momento en que no se enviará ninguna señal a través del canal de comunicación. Durante estos tiempos “muertos” no se envían datos a través del canal. Luego, cuando haya datos para enviar, el dispositivo de envío comenzará a enviar señales, por lo que debe haber una forma para que el dispositivo de destino sepa cuándo comenzar a buscar/leer datos: los dos dispositivos deben establecer y mantener algún tipo de sincronización entre ellos para que las señales se produzcan, transmitan y detecten con precisión. Hay dos alternativas principales para establecer y mantener la sincronización para el muestreo de las señales y se conocen como transmisión asíncrona y síncrona.

En la comunicación asíncrona, la sincronización se restablece con la transmisión de cada carácter mediante el uso de bits de inicio y parada que, dependiendo de la tecnología utilizada, puede haber 1, 1,5 o 2 bits de parada. En la comunicación síncrona, la sincronización se mantiene mediante un bit especial

Característica de transmisión	Serie	Paralela
Descripción	Se envía un bit tras de otro, uno a la vez	Todos los bits de un solo carácter se transmiten simultáneamente
Velocidad comparativa	Más lenta	Más rápida
Limitación de distancia	Más lejos	Más cerca
Aplicaciones	Entre dos computadoras, desde computadoras hasta dispositivos externos, redes de área local (LAN) y amplia (WAN)	Dentro de una computadora a lo largo de los buses de la misma, entre un controlador de unidad y un disco duro
Cables	Todos los bits viajan por un solo cable, un bit a la vez	Cada bit viaja por su propio cable simultáneamente con otros bits

Tabla 2.1: Comparativa entre transmisión paralela vs. serie.

en cada bloque de datos proporcionado por una señal adicional de reloj suministrada por los dispositivos de comunicación. Debido a la menor cantidad de bits utilizados para mantener la sincronización en la comunicación síncrona, esta resulta más eficiente que las comunicaciones asíncronas.

La figura 2.2 muestra de forma esquemática la comunicación serie síncrona y asíncrona entre dos dispositivos. En ambos casos, los dos dispositivos pueden transmitir y recibir información. En la figura 2.2a la comunicación es síncrona, por lo que además de las señales de transmisión y recepción de datos, se incluye también una señal de reloj o clock; mientras que en la figura 2.2b al ser una comunicación asíncrona, esta señal de reloj no es necesaria.

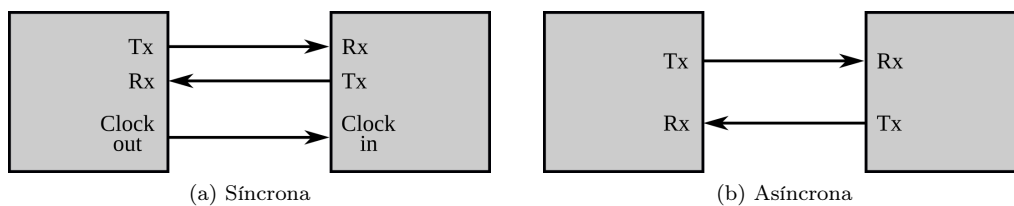


Figura 2.2: Comunicación síncrona vs. asíncrona

En la figura 2.2 “Tx” simboliza el bit de transmisión de datos y “Rx” el bit de recepción.

2.4. Comunicación Half-Duplex y Full-Duplex

Las sesiones de comunicación de datos son en general de naturaleza bidireccional, incluso si el objetivo de la comunicación es enviar un archivo desde el remitente al destino, alguna comunicación debe ir desde el destino de regreso al remitente. Estos datos en dirección inversa incluyen por ejemplo información de corrección y detección de errores. Existen dos maneras de manejar este tipo de tráfico bidireccional: Full-Duplex y Half-Duplex.

En una comunicaciones Full-Duplex, ambos dispositivos pueden transmitir al mismo tiempo, esto es análogo a una conversación en persona. En una Half-Duplex, uno de los dispositivos puede oír o hablar en un momento determinado, lo cual es similar a una conversación que tiene lugar a través de un walkie-talkie (solo puede hablar aquel que presione el botón). De manera similar, cuando dos dispositivos de comunicación de datos se comunican en un entorno Half-Duplex, un dispositivo debe ser el transmisor y el otro el receptor. Para permitir la comunicación bidireccional, periódicamente tienen que invertir roles.

Cuando la comunicación es unidireccional, se conoce con el nombre de Simplex, como es el caso del esquema de la figura 2.1.

2.5. Ejemplos de comunicación paralela y serie

La figura 2.3 muestra la conexión entre una placa Arduino UNO y una pantalla (display) o LCD (Liquid Crystal Display) de caracteres de 16×2 , utilizando 8 bits de datos en paralelo. Estas pantallas se conectan en paralelo utilizando 8 o 4 bits de datos donde, para el último caso, se debe dividir el byte a escribir en 2 nibbles y realizar 2 operaciones de escritura.

Observando de cerca estas pantallas se pueden ver los pequeños rectángulos (píxeles) que forman cada carácter. Cada uno de estos rectángulos es una cuadrícula de 5×8 píxeles. Las pantallas LCD de caracteres vienen de diferentes tamaños y colores: por ejemplo, 16×1 , 16×2 , 20×4 , con texto blanco sobre fondo azul, con texto negro sobre verde entre otras variantes.

Como se observa en la figura, además de los bits de datos se necesitan de otras señales o bits de control, los cuales son: RS (Register Select), E (Enable) y R/W (Read/Write).

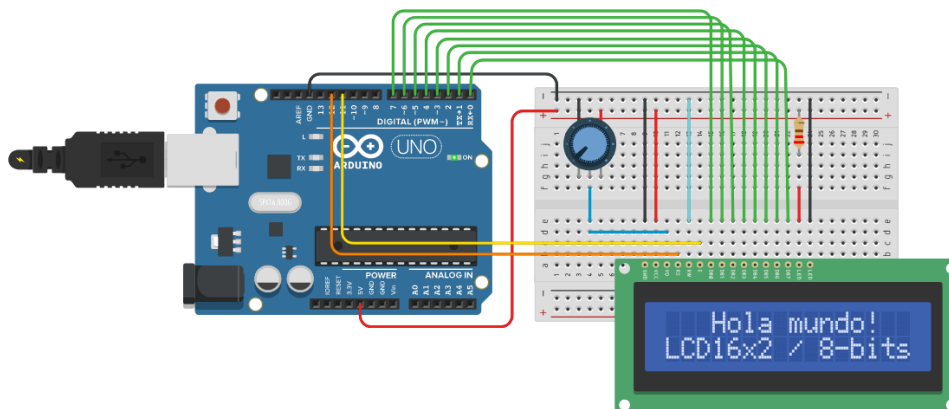


Figura 2.3: Conexión en paralelo de 8-bits entre placa Arduino UNO y LCD de caracteres.

La figura 2.4 muestra la conexión entre una placa Arduino UNO con diferentes módulos con comunicación serie (interfaz UART-TTL). En la figura 3.3a se conecta un módulo de comunicación inalámbrica Bluetooth modelo HC-05 y en la figura 3.3b un módulo GPS (Global Positioning System) modelo NEO-6MV2. Como se verá más adelante, para este tipo de comunicación serie asíncrona se necesitan conectar dos bits de datos (Tx y Rx) además de la referencia común de tensión (GND).

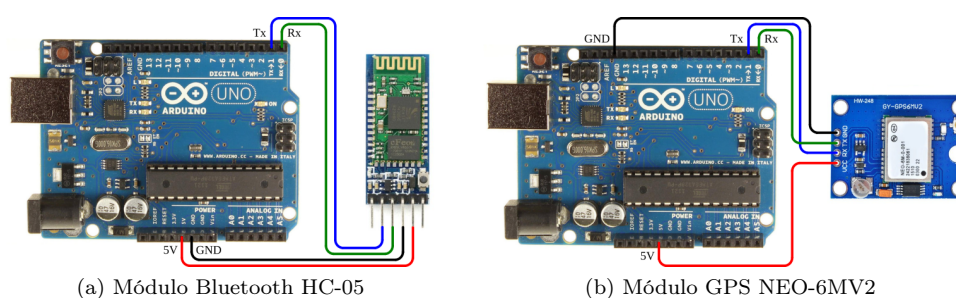


Figura 2.4: Conexión en serie de placa Arduino UNO con módulos Bluetooth y GPS.

Capítulo 3

Estándar RS-232

3.1. Introducción

Debido a su relativa simplicidad en el hardware, en comparación con la interfaz en paralelo, las comunicaciones en serie se utilizan ampliamente en la industria electrónica. Uno de los estándares de comunicación en serie más utilizado es sin duda la especificación EIA/TIA-232-E. Este estándar, que ha sido desarrollado por la *Electronic Industry Association* (Asociación de la Industria Electrónica) y la *Telecommunications Industry Association* (Asociación de la Industria de las Telecomunicaciones), se conoce más popularmente como *RS-232*, donde “RS” significa *Recommended Standard* (Estándar Recomendado).

El nombre oficial del estándar EIA/TIA-232-E es *Interface Between Data Terminal Equipment and Data Circuit-Termination Equipment Employing Serial Binary Data Interchange* (Interfaz entre equipos terminales de datos y equipos de terminación de circuitos de datos que emplean intercambio de datos binarios en serie). Aunque pueda parecer intimidante, el estándar se refiere simplemente a la comunicación de datos en serie entre un sistema host (DTE, Data Terminal Equipment) y un sistema periférico (DCE, Data Circuit-Terminating Equipment o Data Communication Equipment). Gran parte de la terminología RS-232 refleja su origen como estándar para las comunicaciones entre un terminal de computadora (PC) y un módem externo.

Para comprender el uso de la palabra *terminal*, es necesario remontarse al tiempo cuando las computadoras eran del tamaño de habitaciones enteras. Dichas computadoras tenían pocas formas de interactuar con el exterior, las tarjetas perforadas y los rollos de cinta de papel eran una de esas interfaces, pero también existía lo que se conocía como una terminal que se usaba para ingresar y recuperar datos. Estas terminales llegaron en muchos factores de forma pero pronto comenzaron a parecerse a lo que se convertiría en sus descendientes de computadoras personales.

En este sentido, una terminal contiene un teclado, una pantalla, un puerto de comunicación para acceder a una computadora remota y poco más. Un enlace RS-232 conecta el terminal a un módem, que a su vez accede a las líneas telefónicas que se conectan a la computadora remota. Las PC con módems e interfaces de red han hecho que este tipo de conexión de terminal sea casi obsoleto. Además, en la actualidad resulta más frecuente el uso del puerto RS-232 para conectar una PC a un sistema embebido o bien dos sistemas embebidos entre sí.

En el estándar RS-232 se hace referencia al DTE o equipo terminal de datos y DCE o equipo de comunicación de datos, en el cual las señales y sus funciones reciben su nombre desde la perspectiva del DTE. Por ejemplo, Tx (transmisión de datos) es una salida en el DTE y una entrada en el DCE, mientras que Rx (recepción de datos) es una entrada en el DTE y una salida en el DCE. La figura 3.1 muestra la aplicación típica de la interfaz RS-232 en una comunicación utilizando módems.

El RS-232 es un estándar “completo”, lo que significa que el estándar se propone garantizar la compatibilidad entre el host y los sistemas periféricos especificando: 1) niveles comunes de voltaje y señal, 2) configuraciones de pines y cableado y 3) una cantidad mínima de información de control entre el host y los sistemas periféricos.

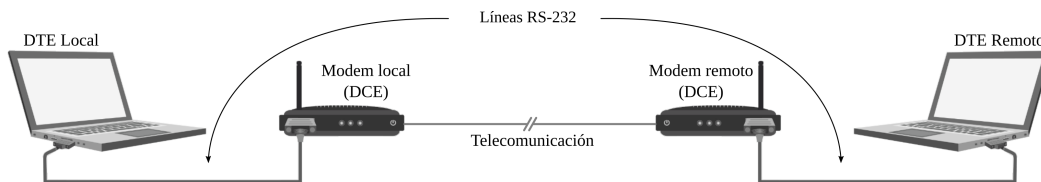


Figura 3.1: Interfaz entre DTE y DCE.

3.2. Características eléctricas

La sección de características eléctricas del estándar RS-232 incluye especificaciones sobre los niveles de tensión, tasa de cambio de niveles de las señales e impedancia de la línea. El estándar RS-232 original se definió en 1962, previo a los días de la lógica TTL (Transistor-Transistor Logic), por lo que no es de extrañar que no utilice niveles lógicos de 5 voltios. En cambio, se define un nivel alto de salida con tensiones entre +5 y +15 voltios y un nivel bajo entre -5 y -15 voltios. Los niveles lógicos de entrada se definieron para proporcionar un margen de ruido de 2 voltios, por lo tanto un nivel alto para la entrada debe estar comprendido entre +3 a +15 voltios y un nivel bajo entre -3 a -15 voltios. Es necesario tener en cuenta que para la comunicación RS-232 un nivel bajo (-3 a -15 voltios) se define como un 1 lógico y se conoce históricamente como *mark* (marca), mientras que un nivel alto (+3 a +15 voltios) se define como un 0 lógico y se denomina *space* (espacio). La figura 3.2 muestra los niveles lógicos definidos por el estándar RS-232.

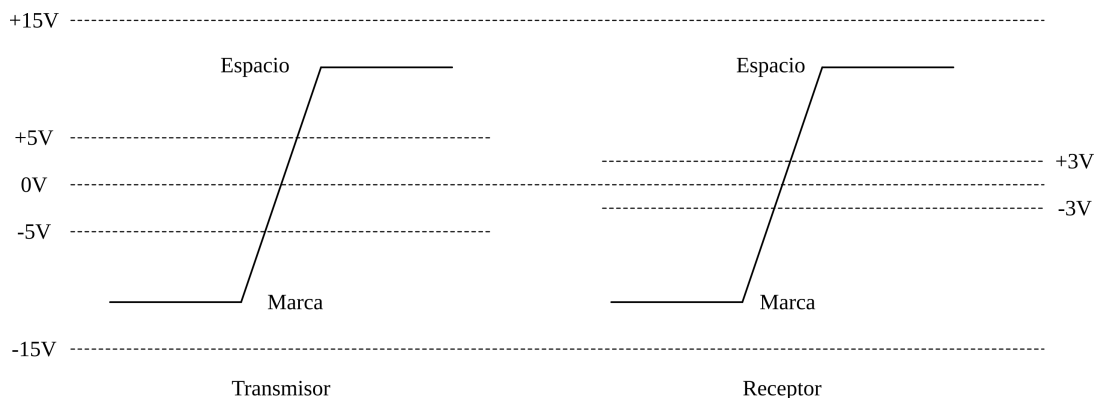


Figura 3.2: Especificaciones de niveles lógicos del RS-232.

El estándar RS-232 también limita la velocidad máxima de respuesta (slew-rate) en la salida de los circuitos para ayudar a reducir la probabilidad de interferencias entre señales adyacentes (cross-talk). Cuanto más lento sea el tiempo de subida y bajada, menor será la posibilidad de interferencia cruzada. Teniendo esto en cuenta, la velocidad de respuesta máxima permitida es de $30\text{V}/\mu\text{s}$. Además, el estándar define una velocidad máxima de datos de 20Kbits/seg., con el propósito también de reducir la posibilidad de interferencias cruzadas.

Para el estándar RS-232 original se especificó una longitud de cable máxima de 15 metros, pero esta parte de la norma se modificó en la revisión "D" (EIA/TIA-232-D), en la cual en lugar de la longitud máxima del cable se especificó una carga capacitiva máxima de 2500pF, que resulta en una especificación más adecuada. La longitud máxima del cable está determinada por la capacitancia por unidad de longitud del cable que se proporciona en las especificaciones propias del cable.

La mayoría de los sistemas actuales no funcionan con niveles de voltaje del estándar RS-232, por lo que se necesita de una conversión de niveles para implementar la comunicación serie, la cual se logra utilizando circuitos integrados especiales. Estos circuitos integrados (CI) suelen tener drivers de salida que generan los niveles de voltaje requeridos por estándar RS-232 y receptores que pueden recibir niveles de voltaje RS-232 sin sufrir daños.

3.3. Características funcionales

El segundo aspecto que está cubierto por el estándar hace referencia a las características funcionales de la interfaz. Esto significa que el RS-232 define la función de las diferentes señales que se utilizan en la interfaz. Estas señales se dividen en cuatro categorías diferentes: referencia, datos, control y sincronización. El estándar proporciona una gran cantidad de señales de control y admite un canal de comunicaciones primario y secundario. Afortunadamente, pocas aplicaciones, si las hay, requieren todas estas señales definidas en el estándar. Por ejemplo, se necesitan de solo ocho señales para un módem típico y algunas aplicaciones sencillas pueden requerir solo cuatro señales (dos para datos y dos para handshake) mientras que otras pueden requerir solo señales de datos sin handshake. El listado completo de las señales definidas por el estándar se puede ver en [16].

Como se mencionó, rara vez se sigue al estándar RS-232 con precisión, debido principalmente a que muchas de las señales definidas no son necesarias para la mayoría de las aplicaciones. Muchas aplicaciones, como un módem, requieren solo nueve señales (dos señales de datos, seis señales de control y referencia). Otras aplicaciones pueden requerir solo cinco señales (dos para datos, dos para protocolo de enlace o handshake y referencia), mientras que otras pueden requerir solo señales de datos sin control de protocolo de enlace. Las señales necesarias para la conexión con un módem y los pines correspondientes del conector DB9 y DB25 se muestran en la tabla 3.1.

# pin DB-9 (DB-25)	Abrev.	Nombre completo	Sentido comun.
3 (2)	TD	Transmitted Data	DTE → DCE
2 (3)	RD	Received Data	DTE ← DCE
7 (4)	RTS	Request To Send	DTE → DCE
8 (5)	CTS	Clear To Send	DTE ← DCE
6 (6)	DSR	Data Set Ready	DTE ← DCE
5 (7)	SG	Signal Ground	DTE — DCE
1 (8)	DCD	Data Carrier Detect	DTE ← DCE
4 (20)	DTR	Data Terminal Ready	DTE → DCE
9 (22)	RI	Ring Indicator	DTE ← DCE

Tabla 3.1: Asignación de pines y sentidos de las señales del RS-232.

3.4. Características mecánicas

El tercer aspecto cubierto por estándar RS-232 hace referencia a la interfaz mecánica. En particular, el RS-232 especifica un conector de 25 pines. Este es el tamaño mínimo del conector que puede albergar todas las señales definidas en la parte funcional del estándar. El conector para el equipo DTE tiene una carcasa hembra con pines de conexión macho, mientras que para el equipo DCE es macho para la carcasa y hembra para los pines. Aunque el RS-232 especifica un conector de 25 pines, el mismo rara vez se utiliza, debido al hecho de que la mayoría de las aplicaciones no requieren todas las señales definidas en el estándar. El conector más popular es el DB9 (9 pines) que se ilustra en la figura 3.3. Este conector resulta adecuado, por ejemplo, para transmitir y recibir las señales necesarias en aplicaciones con módems.

3.5. Receptor/transmisor asíncrono universal, UART

Las señales necesarias para la comunicación en serie son generadas y recibidas por un circuito integrado (CI) conocido como UART (Universal Asynchronous Receiver/Transmitter). El CI que hace de driver de entrada/salida del RS-232 realiza la adaptación de nivel necesaria entre la interfaz CMOS/TTL y RS-232.

La UART realiza las tareas necesarias para la comunicación serial asíncrona la cual, por su naturaleza, es necesario que el sistema host genere los bits de inicio y parada para indicar al sistema periférico cuándo se inicia y termina la comunicación. Los bits de paridad pueden utilizarse también para garantizar que los datos enviados no se hayan dañado. La UART genera los bits de inicio, parada y paridad cuando transmite

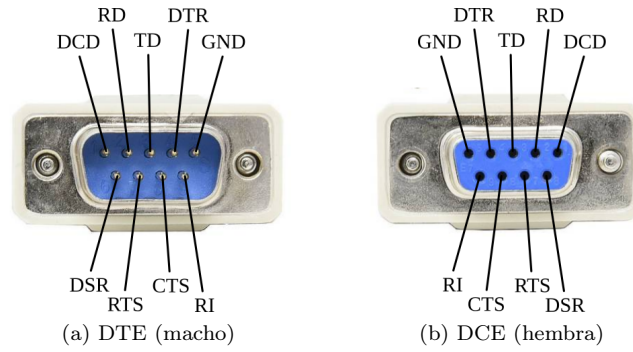


Figura 3.3: Conector DB9 y asignación de pines del RS-232.

datos y puede detectar errores de comunicación al recibir datos. También funciona como intermediario entre la comunicación de a bytes y de a bits, convierte un byte de datos en un flujo de bits en serie en la transmisión y convierte un flujo de bits en serie en un byte de datos en la recepción.

La figura 3.4 muestra la conexión entre la UART, el circuito conversor de niveles de tensión y el conector típico para una comunicación serie con un módem, siguiendo el estándar RS-232.

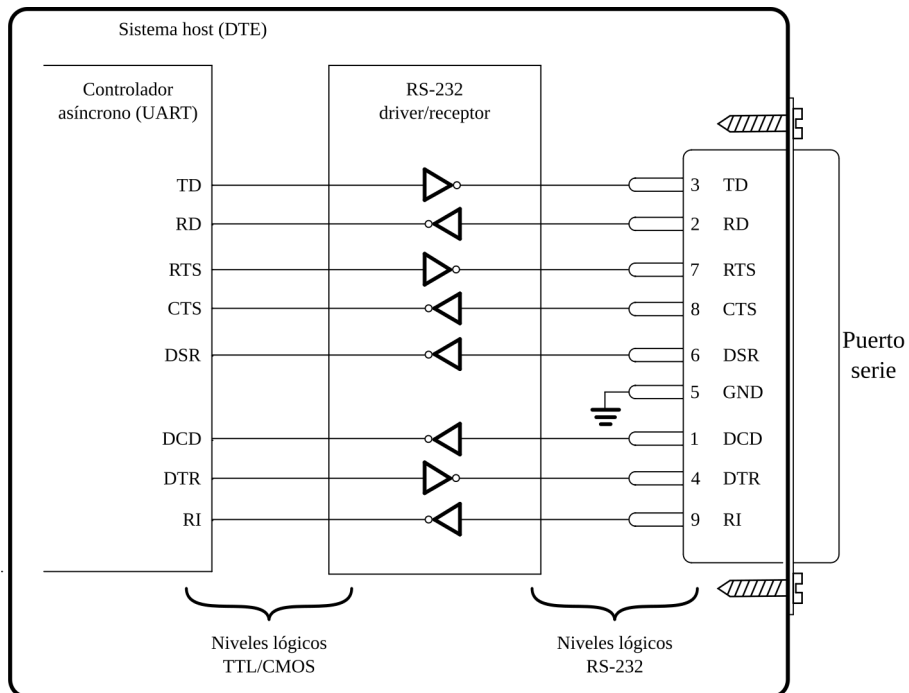


Figura 3.4: Aplicación típica del RS-232 en comunicación con módem.

Capítulo 4

Puerto serie en una PC

4.1. Introducción

Previo a que el USB (Universal Serial Bus) se convirtiera en un estándar de facto como puerto de comunicación de las PC, existían diferentes puertos tales como el PS/2 (que toma su nombre de la serie de computadoras IBM Personal System/2), utilizado particularmente para teclado y mouse, y los antiguos puertos paralelos (IEEE-1284) y seriales (RS-232), utilizados en sus orígenes para la comunicación con impresoras y modems, respectivamente. El puerto paralelo y el serie eran los más adecuados para desarrollar circuitos electrónicos propios que tuvieran la capacidad de comunicarse con la PC.

Aún cuando es común hacer referencia a “puerto serie” como equivalente al estándar RS-232, este no es el único puerto serial usado para comunicar dispositivos electrónicos, como ya se mencionó anteriormente. En particular, en los sistemas embebidos están muy extendido el uso otro tipo de comunicación serie tales como SPI (Serial Peripheral Interface), I²C (Inter-Integrated Circuit), el ya mencionado USB, etc.

Si bien, el puerto de comunicación serie del estándar RS-232 no se encuentran en las PC o laptops actuales, es posible disponer del mismo mediante un adaptador o conversor USB/serie como los que se muestran en la figura 4.1. El adaptador de la figura 4.1a tiene un conector DB9 macho con todas las señales del estándar RS-232, mientras que los de las figuras 4.1b, 4.1c y 4.1d son conversores USB a niveles de tensiones digitales comunes como 5V o 3.3V, los cuales tienen en general las señales TxD, RxD, GND y tensión de alimentación. Estos conversores también se conocen como conversores USB/UART-TTL (Transistor-Transistor Logic). El adaptador de la figura 4.1c tiene el circuito integrado PL2303 de la marca Prolific. Otros de los CIs de uso común en placas de conversión USB/UART-TTL son el CP2101 y CP2102 de Silicon Lab. y el CH340. El adaptador de la figura 4.1d utiliza el CI FT232 de la marca FTDI (Future Technology Devices International Ltd). La diferencia principal de esta placa con las demás es que pone a disposición en sus pines todas las señales de handshake del puerto serie.

4.2. Acceso a dispositivos

En GNU/Linux se tiene acceso a los dispositivos tales como los puertos series mediante archivos de dispositivos, los cuales se encuentran en el directorio `/dev`. Los archivos de dispositivos pueden ser de dos tipos: de caracteres y de bloques (indicados con la letra ‘c’ y ‘b’). En el listado 4.1 se muestran tres archivos de dispositivo: dos de dispositivos de caracteres, `ttyUSB0` y `ttyACM0` de conversores USB/serie; y uno de dispositivo de bloques, el disco `sda`. El tipo de dispositivo está representado por la primer letra en la descripción detallada del archivo a la izquierda del listado, tal como la ‘c’ en `crw-rw----`.

Listado 4.1: Archivos de dispositivos bajo el directorio `/dev`.

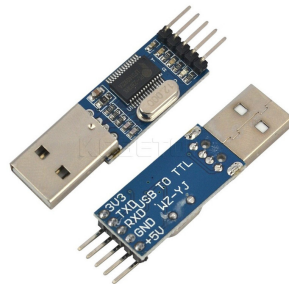
```
crw-rw---- 1 root dialout 188, 0 jul 20 14:02 /dev/ttyUSB0
crw-rw-rw- 1 root dialout 166, 0 jul 20 15:57 /dev/ttyACM0
brw-rw---- 1 root disk     8, 0 jul 20 14:02 /dev/sda
```



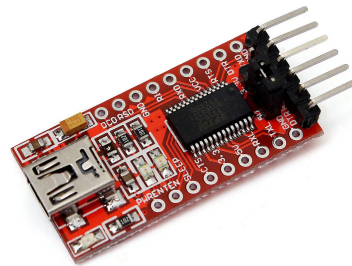

(a) USB a RS-232



(b) USB a UART-TTL con cable



(c) USB a UART-TTL sin cable



(d) USB a UART-TTL con handshake

Figura 4.1: Adaptadores USB/serie.

4.2.1. Dispositivos TTY

El nombre **TTY**, utilizado aún por razones históricas, significa TeleTYpewriter (máquina de escribir remota). Como se mencionó, en los sistemas de tiempo compartidos anteriores a Unix se utilizaban terminales para interactuar con la computadora. Las terminales generaban los mensajes enviados desde un teletipo y la información recibida se imprimía localmente. En estos sistemas las terminales se conectaban de forma remota con la computadora mediante el cable adecuado y una UART.

En los sistemas operativos (SO) modernos como Unix o GNU/Linux se utilizan las terminales para interactuar con el mismo mediante dispositivos TTY. El funcionamiento interno de las terminales lo maneja directamente el Kernel o núcleo del SO. Se puede acceder a las terminales de GNU/Linux presionando las teclas **Ctrl+Alt+Fx**, con **x**: 1, ..., 6. También existen programas emuladores de terminales tales como **gnome-terminal**, **konsole**, **xterm**, **terminator**, entre otras.

Un comando útil para ver y modificar la configuración de la terminal es **stty**. El listado 4.2 muestra la salida del comando **stty** en la terminal actual y el listado 4.3 de la terminal asociada al dispositivo **/dev/ttyUSB0**. La opción **-a** es para imprimir toda la información de configuración de la terminal, mientras que la opción **-F** sirve para indicarle el archivo de dispositivo. Se pueden apreciar los valores de algunos parámetros tales como: longitud de palabra, baudrate, control de flujo, paridad, etc.

Listado 4.2: Configuración de la terminal actual (**/dev/pts/4**).

```
$> stty
speed 38400 baud; line = 0;
-brkint -imaxbel iutf8
```

4.3. Comunicación entre dispositivos

4.3.1. Cable null-model

Para conectar dos dispositivos DTE entre sí a través del puerto serie se puede utilizar un cable especial que recibe el nombre de *null-modem*. Este cable simula una conexión entre un DTE y un DCE dirigiendo cada salida a su entrada correspondiente; por ejemplo, cada salida TD/Tx se conecta a RD/Rx en el otro

Listado 4.3: Configuración del dispositivo serial `/dev/ttyUSB0`.

```
$> stty -a -F /dev/ttyUSB0
speed 9600 baud; rows 0; columns 0; line = 0;
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = <undef>; eol2 = <undef>; swtch
= <undef>; start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W; lnext = ^V;
discard = ^O; min = 1; time = 0;
-parenb -parodd -cmspar cs8 hupcl -cstopb cread clocal -crtcts
-ignbrk -brkint -ignpar -parmrk -inpck -istrip -inlcr -igncr icrnl ixon -ixoff -iucrc -
ixany -imaxbel -iutf8
opost -olcuc -ocrnl onlcr -onocr -onlret -ofill -ofdel nl0 cr0 tab0 bs0 vt0 ff0
isig icanon iexten echo echoe echok -echonl -noflsh -xcase -tostop -echoprnt echoctl echoke
-flusho -extproc
```

extremo del cable o adaptador. El nombre módem nulo se refiere a su origen como un cable que pasa por alto la conexión de computadora a módem (DTE a DCE) y conecta directamente dos computadoras (DTE a DTE). La figura 4.2 muestra varias configuraciones de cable null-modem.

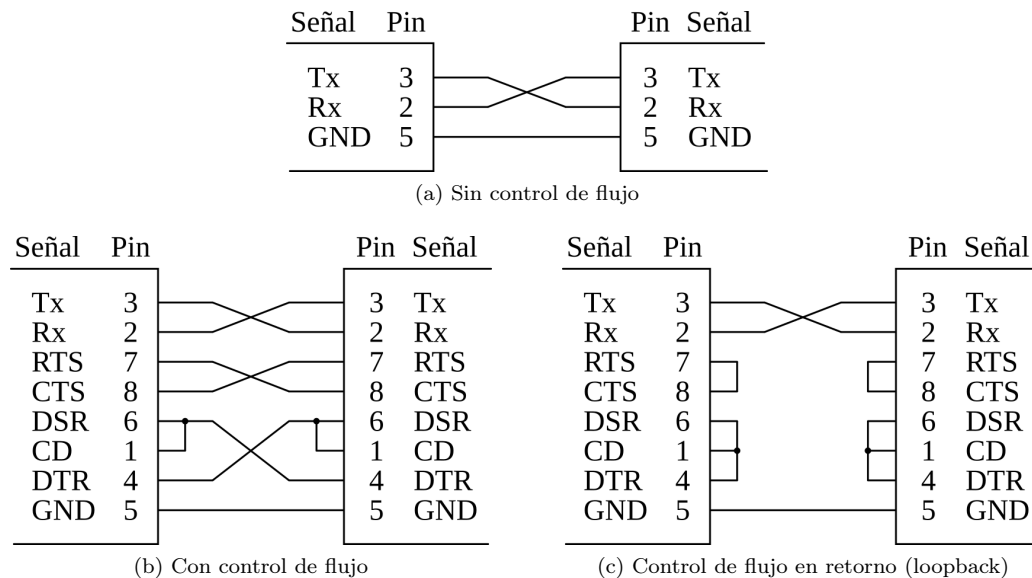


Figura 4.2: Cable null-modem

4.3.2. Utilidad `socat`

`socat` permite generar dispositivos TTY que se comunican entre sí mediante sockets. Es como disponer de dos puertos seriales virtuales conectados entre sí. Por ejemplo, el comando

```
$> socat pty,link=/tmp/ttyS0 pty,link=/tmp/ttyS1
```

crea los archivos de dispositivos TTY, `/tmp/ttyS0` y `/tmp/ttyS1`. Se puede acceder a su configuración mediante `stty -a -F /tmp/ttyS0`. Se puede modificar los parámetros de la comunicación agregando opciones al comando `socat`.

Una vez generados estos archivos y dejando abierta la terminal donde se encuentra ejecutado `socat`, se puede establecer una comunicación entre las dos terminales creadas. Por ejemplo, al ejecutar

```
$> cat /tmp/ttyS1
```

en una terminal, y

```
$> echo "Hola TTY" > /tmp/ttyS0
```

en otra, se estará enviando el mensaje "Hola TTY" desde `ttyS0` a `ttyS1`.

También existen terminales con interfaz gráfica como por ejemplo `cutecom` en el SO GNU/Linux. En la figura 4.3 se muestra una ventana de `cutecom` que tiene abierto la terminal `/tmp/ttyS1` con la

configuración utilizada por `socat` (velocidad de transmisión de 38400 bps, etc). En la parte principal de la ventana se puede ver el mensaje recibido desde `/tmp/ttyS0` generado por el comando `echo`.

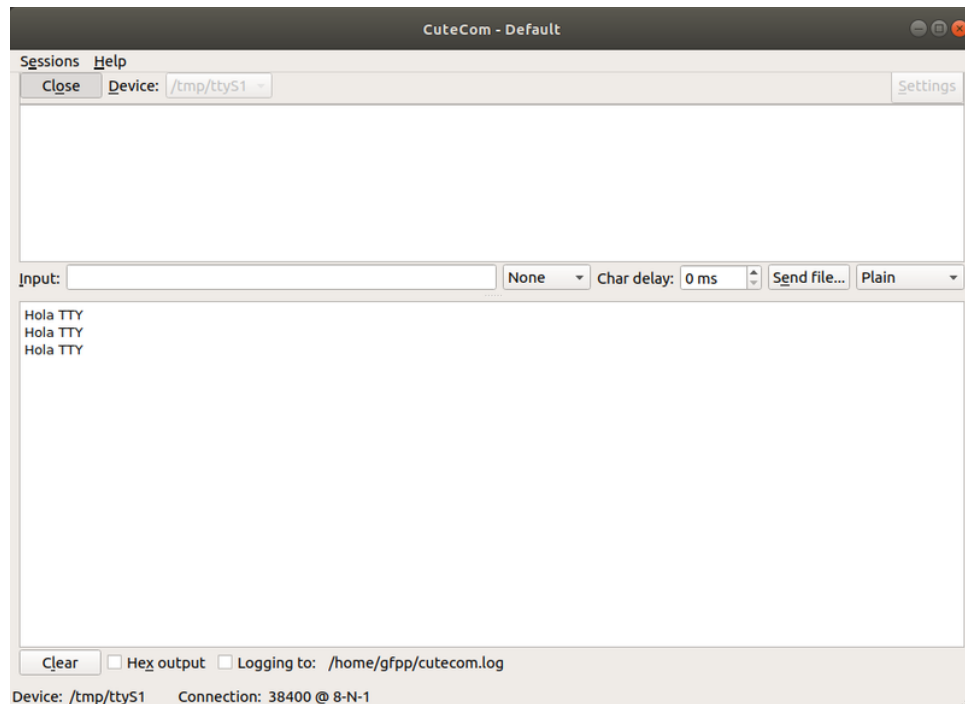


Figura 4.3: Terminal serial gráfica cutecom.

Otros de los programas de terminal o emulador de terminal son: PuTTY que incluye una interfaz gráfica y aplicaciones de consola como `screen`, `minicom`, entre otras.

4.4. Programación del puerto serie de la PC

Como se mencionó anteriormente, dentro del sistema operativo GNU/Linux el puerto serie se puede manejar utilizando el archivo de dispositivo de terminal (TTY) asociado. Para modificar la configuración de la terminal, en particular las opciones propias de la comunicación (trama de datos, velocidad de comunicación, control de flujo, etc.), se utilizan funciones de bibliotecas separadas de aquellas utilizadas para la lectura y escritura.

4.4.1. Programa ejemplo

En el listado 4.4 se muestra el código fuente para escribir un mensaje en el archivo asociado a un dispositivo serie, en este caso en `/tmp/ttyS0`. En este programa, primero se abre el archivo con la función `open()`, luego se escribe al archivo con la función `write()` y finalmente se cierra con la función `close()`. Los argumentos de la función `open()` son: el archivo (con su ruta absoluta) y las banderas (flags) que indican las propiedades al abrir el archivo. En el ejemplo la bandera `O_WRONLY` abre el archivo para solo escritura. En este ejemplo no se modifica la configuración de la terminal por lo que se utiliza la configuración que tiene por defecto.

El código fuente del listado 4.4 se puede compilar con

```
$> gcc -Wall writetty.c -o writetty
```

lo que genera el archivo binario `writetty`. Este se puede ejecutar con

```
$> ./writetty
```

Al ejecutar el programa se puede ver el mensaje enviado con `cutecom`, como en la figura 4.3.

Listado 4.4: Escritura en un archivo de dispositivo serie, `writetty.c`.

```
1 #include <stdio.h>
2 #include <unistd.h> /* Unix standard function definitions */
3 #include <fcntl.h> /* File control definitions */
4
5 int main(void)
6 {
7     int fd; /* File descriptor */
8
9     fd = open("/tmp/ttyS0", O_WRONLY);
10    if(fd == -1)
11    {
12        printf("ERROR: no se pudo abrir /tmp/ttyS0\n");
13        return -1;
14    }
15
16    write(fd, "Hola_TTY\n", 9);
17    close(fd);
18    return 0;
19 }
```

4.4.2. Configuración de la terminal – estructura `termios`

`termios` es una interfaz especificada por el estándar POSIX (Portable Operating System Interface). La interfaz de terminal se controla fijando valores a un tipo estructura `termios` y utilizando un pequeño conjunto de funciones, ambas declaradas en `termios.h`. Los valores que se pueden manipular para afectar el comportamiento de la terminal están agrupados en varios modos: entrada, salida, control, local y caracteres especiales. La definición mínima de estructura `termios` se muestra en el listado 4.5, donde los nombres de los miembros se corresponden con los tipos de parámetros ya mencionados.

Listado 4.5: Estructura `termios` definida en `termios.h`.

```
1 struct termios {
2     tcflag_t c_iflag;
3     tcflag_t c_oflag;
4     tcflag_t c_cflag;
5     tcflag_t c_lflag;
6     cc_t c_cc[NCCS];
7 };
```

Se puede inicializar una estructura `termios` desde la configuración actual llamando a la función `tcgetattr()`, como

```
int tcgetattr(int fd, struct termios *term);
```

lo cual carga los valores actuales de configuración de la terminal a la estructura apuntada por el parámetro `term`. A partir de aquí se pueden cambiar los valores de los campos de la estructura y luego configurar la terminal con los valores modificados utilizando la función `tcsetattr()`.

```
int tcsetattr(int fd, int actions, const struct termios *term);
```

El parámetro `actions` controla cómo se aplican los cambios¹, pudiendo ser:

- **TCSANOW**: cambiar los valores inmediatamente.
- **TCSADRAIN**: cambiar los valores cuando la salida actual este completa.
- **TCSAFLUSH**: cambiar los valores cuando la salida actual este completa, pero descartando cualquier entrada disponible y que no ha sido leída aún por una llamada a `read()`.

4.4.3. Módulo de software para configuración del puerto serie

Para poder utilizar una función de configuración de puerto serie en diferentes aplicaciones es conveniente tener un módulo de software separado de dicha implementación². Este módulo está comprendido

¹Ver página de manual: `man 3 tcsetattr`

²Un módulo de software es un conjunto de archivos `.h` y `.c`

por el archivo de cabecera del listado 4.6 con la declaración de la función de configuración y el archivo de código fuente del listado 4.7 con la correspondiente implementación. En esta implementación, la función `termset` configura la comunicación serie con la trama más común de comunicación 8N1 (8 bits de datos, sin bit de paridad y 1 bit de stop), habilitado para lectura y sin control de flujo por hardware ni por software (XON/XOFF).

Listado 4.6: Archivo de cabecera del módulo, `termset.h`.

```
1 #ifndef TERMSET_H
2 #define TERMSET_H
3
4 #include <termios.h>
5
6 struct termios ttyold, ttynew;
7
8 /* termset function
9  * Parameters:
10  * fd: file descriptor -device- (ex: /dev/ttyUSB0)
11  * baudrate: communication speed (ex: 9600, 115200)
12  * ttyold: current termios structure
13  * ttynew: new termios structure
14  */
15 int termset(int fd, int baudrate, struct termios *ttyold,
16            struct termios *ttynew);
17
18 #endif
```

Listado 4.7: Archivo de código fuente del módulo, `termset.c`.

```

1 #include <stdio.h>
2 #include "termset.h"
3
4 int termset(int fd, int baudrate, struct termios *ttyold,
5             struct termios *ttynew)
6 {
7     switch(baudrate)
8     {
9         case 115200: baudrate = B115200;
10                    break;
11         case 57600:  baudrate = B57600;
12                    break;
13         case 38400:  baudrate = B38400;
14                    break;
15         case 19200:  baudrate = B19200;
16                    break;
17         case 9600:   baudrate = B9600;
18                    break;
19         default:     baudrate = B115200;
20                    break;
21     }
22
23     if(tcgetattr(fd, ttyold) != 0)
24     {
25         printf("ERROR: \tcgetattr\n");
26         return -1;
27     }
28     ttynew = ttyold;
29
30     cfsetospeed(ttynew, baudrate);
31     cfsetispeed(ttynew, baudrate);
32
33     ttynew->c_cflag = (ttynew->c_cflag & ~CSIZE) | CS8; // 8 data bits (8)
34     ttynew->c_cflag &= ~(PARENB | PARODD);           // no parity (N)
35     ttynew->c_cflag &= ~CSTOPB;                       // 1 stop bit (1)
36
37     ttynew->c_cflag |= (CLOCAL | CREAD); // ignore modem status lines, and
38                                     // enable reading
39     ttynew->c_cflag &= ~CRTSCTS;         // no flow control
40
41     ttynew->c_iflag &= ~IGNBRK;          // disable break processing
42     ttynew->c_iflag &= ~(IXON | IXOFF | IXANY); // shut off xon/xoff ctrl
43
44     ttynew->c_lflag = 0;                 // no signaling chars, no echo,
45     ttynew->c_oflag = 0;                 // no remapping, no delays
46     ttynew->c_cc[VMIN] = 0;              // read doesn't block
47     ttynew->c_cc[VTIME] = 100;           // read timeout
48
49     /*
50      * TCSANOW: Make the change immediately.
51      * TCSADRAIN: Make the change after all queued output has been written.
52      * TCSAFLUSH: This is like TCSADRAIN, but also discards any queued input.
53      */
54     if(tcsetattr(fd, TCSAFLUSH, ttynew) != 0)
55     {
56         printf("ERROR: \tcsetattr\n");
57         return -1;
58     }
59
60     return 0;
61 }

```

Bibliografía

- [1] Getting Started with Arduino. <https://docs.arduino.cc/learn/starting-guide/getting-started-arduino/>.
- [2] Atmega328 datasheet. <https://www.microchip.com/en-us/product/ATMEGA328P>.
- [3] CIII, UTN-FRC. PonchitoCIII. <https://github.com/ciiiutnfr/ponchitoCIII>.
- [4] CIII, UTN-FRC. PonchitoCIII en TinkerCAD. <https://www.tinkercad.com/things/a9nrlsl5pPs-ponchitociii>.
- [5] Arduino web page. <https://www.arduino.cc/>.
- [6] Hernando Barragán. The untold history of arduino. <https://arduinohistory.github.io/>.
- [7] Arduino uno rev3. <https://store.arduino.cc/products/arduino-uno-rev3>.
- [8] Gonzalo Perez-Paina. Repositorio atmega328. <https://github.com/gfpp/atmega328>.
- [9] Autodesk. TinkerCAD. <https://www.tinkercad.com/>.
- [10] Processing web page. <https://processing.org/>.
- [11] Arduino IDE. <https://www.arduino.cc/en/software>.
- [12] Arduino libraries reference. <https://docs.arduino.cc/language-reference/>.
- [13] Avrgcc. <https://gcc.gnu.org/wiki/avr-gcc>.
- [14] AVR libc home page. <https://nongnu.org/avr-libc/>.
- [15] Gcc avr options. <https://gcc.gnu.org/onlinedocs/gcc/AVR-Options.html#AVR-Options>.
- [16] Dallas Semiconductor. Application Note 83. Fundamentals of RS-232 Serial Communications.